

Sistema Inteligente de Detección, Identificación y Análisis de Vehículos

Rubén Fernández González
rfernng19@estudiantes.unileon.es

May 17, 2026

Abstract

Esta es una propuesta de investigación sobre el diseño e implementación de una prueba de concepto de un sistema inteligente para la detección, identificación y análisis de vehículos en entornos reales (cámaras embarcadas y puntos fijos como peajes). El sistema integrará técnicas de visión por computador y de aprendizaje profundo para detectar e identificar vehículos en vídeo en tiempo real.

Keywords: *Detección de vehículos, Identificación, Deep Learning, Cámaras embarcadas*

1 Introducción

La detección e identificación de vehículos en tiempo real es una tarea muy importante actualmente, debido al crecimiento de la industria de coches autónomos (y otros también no autónomos), donde unos sensores se encargan de detectar otros vehículos en las proximidades para poder razonar y actuar gracias a esta información, como pueden ser unos sensores en distintas partes del vehículo para que, al aparcar, puedan detectar y medir distancias a un vehículo próximo. Y no solo es importante para los coches autónomos también lo es para:

- **Control del tráfico:** Se puede implementar un programa para que detecte y cuente la cantidad y tipo de vehículos de una intersección para variar dinámicamente el estado de los semáforos.
- **Coches robados:** Simplemente se puede hacer una detección de los coches y compararlos uno a uno con una base de datos de coches, o vehículos en general, robados y poder realizar avisos a la policía.

La detección y reconocimiento precisa de un vehículo no es una tarea sencilla, incluso para los humanos diferenciar entre muchas marcas existentes y aún más, entre los diferentes modelos de cada una de las marcas, es una tarea sumamente compleja. Las redes neuronales permiten hacer esta tarea de forma eficaz y precisa, pudiéndose comunicar con otro tipo de programas de señales o emergencias, por ejemplo.

En general, la mayoría de investigadores usan alguno de los tantos modelos YOLO existentes. Ya que YOLO destaca en cualquier tipo de tarea gracias a su flexibilidad, es especialmente prominente a dar buenos resultados en la tarea de detección y reconocimiento de vehículos. Un ejemplo claro puede ser el caso del proyecto de **Sarvesh Shirulkar et Al.** [1] con YOLO para controlar las señales de tráfico según las detecciones del modelo YOLO. Otro ejemplo es el de **Anning Ji y Xintao Ma** [2], en el que utilizan como base un modelo YOLO10 al que se le modifica la parte de extracción de características. Aunque estos modelos predominen la elección de investigadores, hay otras opciones con diferentes

arquitecturas que pueden ser muy útiles en determinados escenarios, como puede ser la red neuronal de **Leyre Encio et Al.** [3] basada en CenterNet, que trata de asimilar un diseño similar a la de los transformadores con su *encoder-decoder*.

En la sección 2 o **Estado del arte**, se nombran varios métodos, principalmente con modelos YOLO, para detectar y reconocer todo tipo de vehículos en distintos entornos. Los modelos (o métodos) seleccionados se explicarán detalladamente en la sección 3 o **Métodos seleccionados**. En la sección 4 o **Datasets** se disponen los conjuntos de datos de vehículos que se van a utilizar para entrenar y probar los modelos.

2 Estado del arte

2.1 Detección de vehículos

K. Saiveena y P. Praaven [4] tienen un enfoque de detección y segmentación de vehículos basado en *Edge Intelligence* (EI), una rama de *Deep Learning*. El programa, **ViT-YOLOv5**, se divide en dos partes fundamentales: la primera es un ViT (*Vision Transformer*), encargado de **extraer y procesar las características** de las imágenes sustituyendo la labor del extractor de características del detector **YOLOv5**¹: **CSPDarknet**. Esta integración permite capturar de manera más eficiente tanto las características globales como las locales. La segunda parte consta del detector **YOLOv5**, que se encarga de generar las predicciones de clase, su confianza y las *bounding boxes*. Para medir el rendimiento de este proyecto se obtuvieron 4 datasets de vehículos (dirigidos a detección de objetos principalmente) de la web de Kaggle², alcanzando un *accuracy* de **97.25%**, **94.42%**, **94%** y **95.7%** para cada dataset, respectivamente. Un análisis profundo con diferentes configuraciones y modelos se puede observar en la figura 1, con diferencias notables entre el **YOLOv5** base y el modificado con el transformador, **ViT-YOLOv5**. Este modelo se ha entrenado en una NVIDIA Tesla V100 GPU de 32GB de RAM.

Terms	Yolov4 [18]	Edge Yolo [19]	Yolov3 [22]	Yolov5 [48]	ViT-YOLOv5
Dataset 1					
Linear	77.75	92	78.5	91.66666667	94.33333333
ReLU	82	90.58333333	82.83333333	95.25	97.25
TanH	84.41666667	86	80.16666667	90.91666667	96
Softmax	75.58333333	90.5	80.5	93.83333333	96.08333333
Sigmoid	77.16666667	91.75	91.25	94.91666667	95.5
Dataset 2					
Linear	77.91666667	91.33333333	85.66666667	88	93.41666667
ReLU	80.83333333	84.66666667	86	89.91666667	94.41666667
TanH	85.75	83.75	87.25	88.33333333	93.5
Softmax	76.58333333	86.08333333	81.08333333	86.41666667	93.41666667
Sigmoid	84.08333333	89.91666667	87.25	91.16666667	96.25
Dataset 3					
Linear	84.25	81.5	83.5	90.33333333	95.83333333
ReLU	75.5	82	89.58333333	84.41666667	94
TanH	78.58333333	82.5	77.66666667	89.66666667	94.91666667
Softmax	86.83333333	82.25	80.58333333	84.08333333	97.66666667
Sigmoid	85.66666667	85.66666667	77.33333333	90.58333333	95.16666667

Figura 1: Rendimiento de diferentes modelos y configuraciones en los datasets de vehículos

Fuente: <https://www.sciencedirect.com/science/article/pii/S0045790626001588>

La identificación precisa de objetos en la gestión del tráfico urbano y el transporte

¹Accesible a través de GitHub: <https://github.com/ultralytics/yolov5>

²Accesibles a través de: Sakshamjn, Saumyapatel, Seyeon y Nadinpethiyagoda

autónomo es clave y, por eso, Wenjie Zhu et Al. [5] proponen el modelo **DFEM-Net**³ que **mejora la flexibilidad** al aprender diversas formas, tamaños y detalles superficiales, algo que es muy importante a la hora de realizar este tipo de tareas. El primer cambio es el **ajuste dinámico del tamaño** y de la forma de los kernels convolucionales a través de una **TuaNet**: una integración de una **DCN** (*Deformable Convolutional Network*) junto con **Tua Attention**. Además, se introduce un módulo de fusión de características multi-escala **Scalseq**, que ayuda a la red a **extraer características a distintos niveles**. Al mismo tiempo, un módulo **Zoomcat** de triple cifrado de características **reduce el solapamiento** para superponer pequeños objetos. Este método obtiene un **77.5%** de mAP@0.5 y un **59.3%** de mAP@0.5:0.95 en el **TVAP-Normal Dataset**, y un **71.3%** de mAP@0.5 y un **46.5%** de mAP@0.5:0.95 en el **TVAP-Foggy Dataset**. En comparación, YOLOv8 obtiene un 74.6% de mAP@0.5 y un 57.6% de mAP@0.5:0.95 en el TVAP-Normal dataset, algo que se puede observar con 4 imágenes del dataset en la figura 2. El entrenamiento se ha realizado durante 400 iteraciones con un optimizador SGD y una tasa de aprendizaje inicial de 0.01 con un decaimiento de 0.0005 en una Tesla V100-SXM2-16GB GPU. Además, **DFEM-Net** mantiene una alta eficiencia computacional con **9.1 GFLOPs**, comparable con los 8.7 GFLOPs de la versión más pequeña de YOLOv8.



Figura 2: Resultados de YOLOv8 (izquierda) y DFEM-Net (derecha) en 4 distintas imágenes del dataset TVAP-Normal

Fuente: <https://www.sciencedirect.com/science/article/pii/S026322412600045X>

Un framework unificado y optimizado de detección, tracking, conteo, predicción de movimiento y clasificación de vehículos es presentado por Mohammed Alnusayri et Al. [6]. El framework consta de **YOLOv10**, **ByteTrack**, **Optical-flow density estimation**, predicción de trayectorias con **Long Short-Term Memory-based (LSTM-based)** y extracción de características con **Speeded-Up Robust Feature (SURF)** y **Gray-Level Co-occurrence Matrix (GLCM)** junto al modelo **VGG16**. Los conjuntos de datos utilizados son **UAVid** y **UAVdt**⁴, cuyas imágenes están a una altura de unos 50-70 metros de altura, en los que el framework alcanza una precisión en la detección de **95.49%**, **95.15%** en recall y **95.52%** en f1-score.

³El modelo es completamente accesible a través de <https://github.com/ahtarv/DFEM-Net>

⁴Los datasets son accesibles por UAVID y UAVDT

Algunos ejemplos se pueden ver sobre la figura 3. Las pruebas se han realizado sobre una Intel i7-12700K (3.6 GHz), 32 GB RAM y una NVIDIA RTX 3090 GPU.



Figura 3: Ejemplo de detección de vehículos sobre una vista UAV
Fuente: <https://www.sciencedirect.com/org/science/article/pii/S1546221826000974>

La detección de objetos pequeños y condiciones de tráfico son un desafío para la gestión del tráfico, así que Anning Ji y Xintao Ma [2] desarrollan un framework de detección muy novedoso: **YDFNet**, que integra **YOLOv10** para extraer rápidamente las características, una **BiFPN** (*Bidirectional Feature Pyramid Network*), cuya arquitectura se puede observar en la figura 4, para fusionar las características a distintas escalas y un transformador de detección **DETR**. Este framework utiliza las ventajas de cada una de las partes para mejorar la eficiencia y precisión de la detección. Para experimentar, se escogen los datasets **UA-DETRAC** y **COCO**⁵, en los que alcanza un **75.8%** de mAP y un **52.2%** de AP_{small} mejorando resultados anteriores como los de YOLOv10 en los mismos datasets: 67.8% de mAP y un 42.5% de AP_{small} . Las pruebas fueron realizadas en entornos con restricciones de recursos a través de NVIDIA Jetson AGX Xavier and Rockchip RK3588.

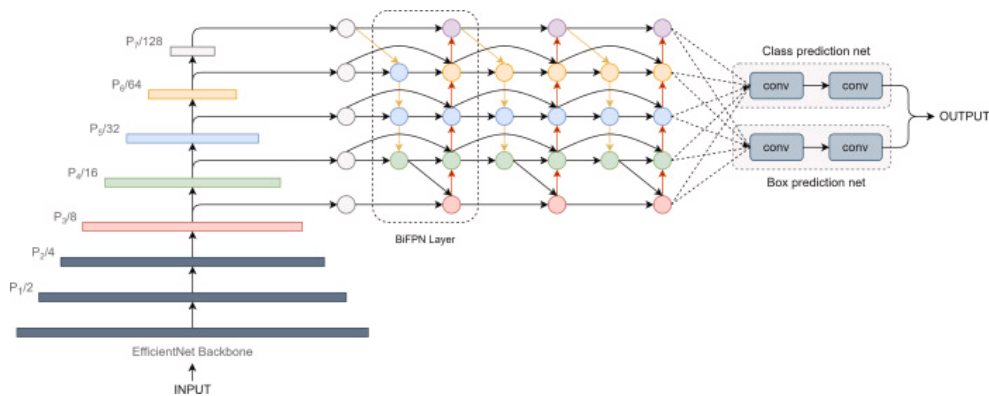


Figura 4: Arquitectura de una FPN bidireccional (BiFPN)
Fuente: <https://www.sciencedirect.com/science/article/pii/S1110016825007999>

Otro avance necesario en la gestión del tráfico es el de anomalías, Seyed H.H.D. et Al. [7]

⁵Los datasets son obtenibles por UA-DETRAC y COCO

proponen un sistema en tiempo real de detección de vehículos, anomalías de tráfico y comportamiento de los conductores. El sistema integra datos geoespaciales y del tiempo para adaptarse a condiciones cambiantes. Los modelos utilizados son **YOLOv8** y **YOLOv11**. Las imágenes fueron adquiridas a través de varios datasets de imágenes de tráfico en determinadas áreas de tráfico de la provincia de Isfahan, Irán. Además, las imágenes fueron adquiridas desde distintos ángulos para replicar las perspectivas de las cámaras de vigilancia. Los resultados obtenidos para cada uno de los modelos, en cuestión de tanto detección de vehículos como de anomalías, se pueden observar en la tabla 1.

Modelo	Precision	Recall	mAP50	mAP50-95
YOLOv8	86.1%	73.0%	87.4%	68.2%
YOLOv11	89.7%	72.3%	81.3%	62.4%

Tabla 1: Resultados de YOLOv8 y YOLOv11 en entornos de tráfico

En cámaras de monitorización estáticas, las imágenes son tomadas automáticamente: sin tener la garantía de que los objetos de interés (en este caso los vehículos) se puedan apreciar con claridad, ya sea por clima, la localización de la cámara o que el objeto es parcialmente observable. Para resolver este problema, Sara Beery et Al. [8] deciden crear una CNN con un módulo de atención de forma que se aprovecha el contexto temporal de los frames no etiquetados para mejorar la precisión, **Context R-CNN**. En este modelo, donde la base es una **Faster R-CNN**, las bounding boxes candidatas de la primera parte primero se pasan a través de 2 módulos de atención que indexan en dos depósitos de memoria para **proveer contexto local y global**. Estos módulos devuelven un vector de características que se pasa a la segunda parte del modelo **Faster R-CNN** de forma normal.

En la figura 5 se puede ver el modelo completo. Esta propuesta alcanza un **42.6%** de mAP en el dataset CityCam⁶.

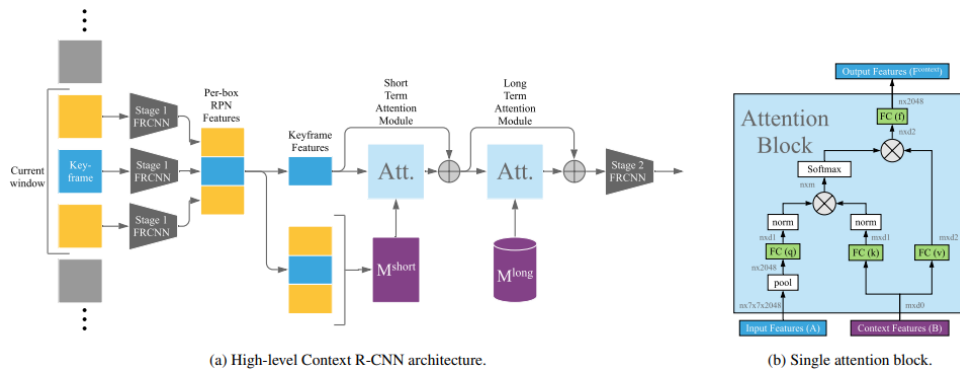


Figura 5: Arquitectura de una Context R-CNN

Fuente: <https://arxiv.org/pdf/1912.03538>

La gestión automática de la entrada y salida de vehículos en aparcamientos es un desafío en temas de eficiencia y seguridad. Para solucionar esto, Muhammad U.R. et Al. [1] utilizan modelos de vanguardia para la detección del vehículo, verificación de la matrícula,

⁶El dataset es accesible por CityCam

y detección y reconocimiento del conductor; el esquema se puede observar en la figura 6. Después de probar varios modelos, el que más destaca en cada uno de los anteriores apartados es **YOLOv8n**, la versión más pequeña de **YOLOv8**, destacando por encima de **YOLOv5** y **Faster R-CNN**. Ya que se utilizan 2 modelos para las dos primeras tareas de detección (vehículo y matrícula por separado), se necesitan conjuntos de datos diferentes: para vehículos se recogieron 6000 imágenes de 3 categorías diferentes y para las matrículas se recogen otras 600 de matrículas de coches y bípedos. Para el reconocimiento de rostros se usa *Haar cascades*. El sistema alcanza un **99.2%** de precisión, **99%** de recall, **99.4%** de mAP@50 y **99.1%** de IoU, y **99.1%** de precisión, **99.1%** de recall, **99.2%** de mAP@50 y **98.2%** de IoU en las tareas de detección de vehículos y detección de matrículas, respectivamente.

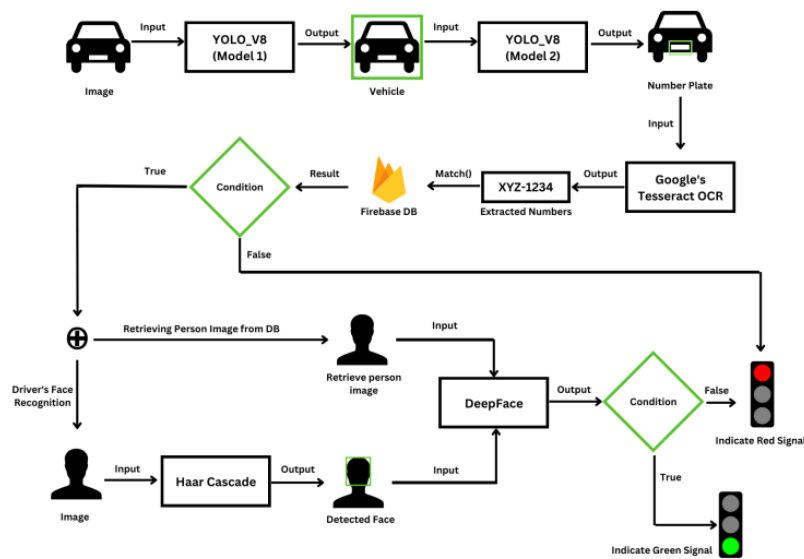


Figura 6: Esquema de flujo de la tarea de detección de vehículos y de matrículas

Fuente: <https://arxiv.org/pdf/2312.02699>

La congestión del tráfico es un grave problema para países superpoblados, sobre todo ciudades, como en la India. Sarvesh Shirulkar et Al. [9] idean un sistema adaptativo para controlar las señales de tráfico en intersecciones usando detección y seguimiento de vehículos en tiempo real con el algoritmo **YOLO**, reduciendo de gran manera la congestión del tráfico y mejorando la circulación urbana. El funcionamiento de este sistema consta de 5 pasos fundamentales:

- Primero se captura la imagen inicial de la cámara de tráfico.
- Con YOLO se cuentan los vehículos de cada tipo: camiones, autobuses, coches y motocicletas no tienen el mismo tamaño.
- Se calcula la densidad del tráfico, teniendo en cuenta un peso asignado a cada categoría y su influencia en la circulación.
- Se modifica dinámicamente el estado y la temporización de la señal verde del semáforo. El resultado se puede observar en la figura 7.



Figura 7: Resultado del sistema con el conteo de vehículos y el temporizador del semáforo

Fuente: <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10984828>

Este siguiente proyecto de B. Shyamala Gowri et Al. [10] desarrolla un sistema de gestión del tráfico que integra la versión **YOLOv8** para la detección de vehículos y una priorización de las señales de tráfico con **GNNs (Graph Neural Networks)** para solucionar los problemas de los sistemas de tráfico tradicionales. Estos sistemas fallan a la hora de detectar los vehículos de emergencia y adaptar las señales de tráfico según convenga para ayudar a la circulación. El sistema propuesto junta las capacidades de detección de **YOLO** con el aprendizaje reforzado de las **GNNs** para ajustar dinámicamente las señales de tráfico en tiempo real según las condiciones de la carretera. Para el conjunto de datos, se obtuvieron grabaciones de tráfico de cámaras de coches (dashcams), UAVs y cámaras de vigilancia. La detección alcanzó un **94.7%** de precisión, **93.5%** de recall, **94.1%** de f1-score y **9.6 ms** de tiempo de detección en el conjunto de pruebas, una mejora considerable como se puede ver en la figura 8 comparándolo con otros métodos como **YOLOv5** y **Faster R-CNN**.

Otro país con gran densidad en las ciudades es China, donde cada vez los atascos son mas prolíficos a suceder, afectando la vida de las personas y su eficiencia en el trabajo. Para solucionar esto, Xiaohui Guan y Xinxin Sun [11] presentan un método de detección de vehículos y estadísticas de tráfico apropiado para cualquier tipo de intersecciones y cruces. Para poder tener un buen modelo entrenado, se extraen únicamente 4 categorías (coches, autobuses, motocicletas y camiones) del dataset **COCO2017** y todas las imágenes se normalizan a una dimensión de 299 x 299 píxeles. En el entrenamiento con validación cruzada se usa el modelo de detección **YOLOv3**, con una tasa de aprendizaje de **1e-4**. Además de probar el modelo de detección con un vídeo, se utiliza el modelo de seguimiento **Deep-Sort** para predecir la localización de los vehículos en el siguiente fotograma. Por último, se

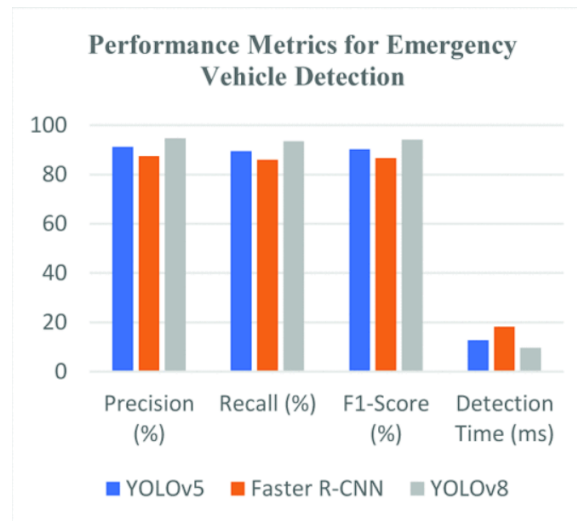


Figura 8: Rendimiento de YOLOv8 frente a otros detectores en detección de vehículos
Fuente: <https://ieeexplore.ieee.org/document/10988348>

contabiliza el flujo del tráfico. En el vídeo de pruebas, la intersección se divide en 5 áreas según la figura 9, consiguiendo que el modelo **YOLO** detecte 472 vehículos en total, donde sólo 9 de ellos se contaron erróneamente.



Figura 9: Sistema de detección de vehículos con intersecciones
Fuente: <https://ieeexplore.ieee.org/document/9858960>

Para solucionar los problemas de detección de vehículos en entornos complejos, Chenyu Gu et Al. [12] investigan y diseñan un proyecto para atajar este problema. El primer paso es construir un dataset que incluya entornos complejos e imágenes con vehículos pequeños. El modelo seleccionado es **YOLOv7**, o al menos la base, ya que integra un canal multi-escala y un mecanismo de atención especial: **Coordinate Attention (CA)**. La estructura completa de este algoritmo modificado, **YOLOv7-veh**, se muestra en la figura 10, donde el mecanismo **CA** funciona como una unidad computacional para **mejorar la representación**

de características, recibiendo características de capas intermedias y mejorando esas mismas características sin cambiar la dimensión. Para la experimentación se utilizó el dataset **BIT-Vehicle**⁷, el cual contiene 9850 imágenes de vehículos capturadas en diferentes tiempos y escenarios. Debido a las limitaciones de hardware, se construyó un conjunto más pequeño de 4800 imágenes (proporción de entrenamiento/pruebas de 7:3), alcanzando un **96.2%** de mAP@0.5 y **18.9 FPS**, mejorando los 21.7 FPS y 93.5% de mAP@0.5 del YOLOV7 sin modificar. Las pruebas se han realizado sobre una Intel i9-13900HX CPU de 32 núcleos y una NVIDIA RTX 4090 GPU (12GB de VRAM).

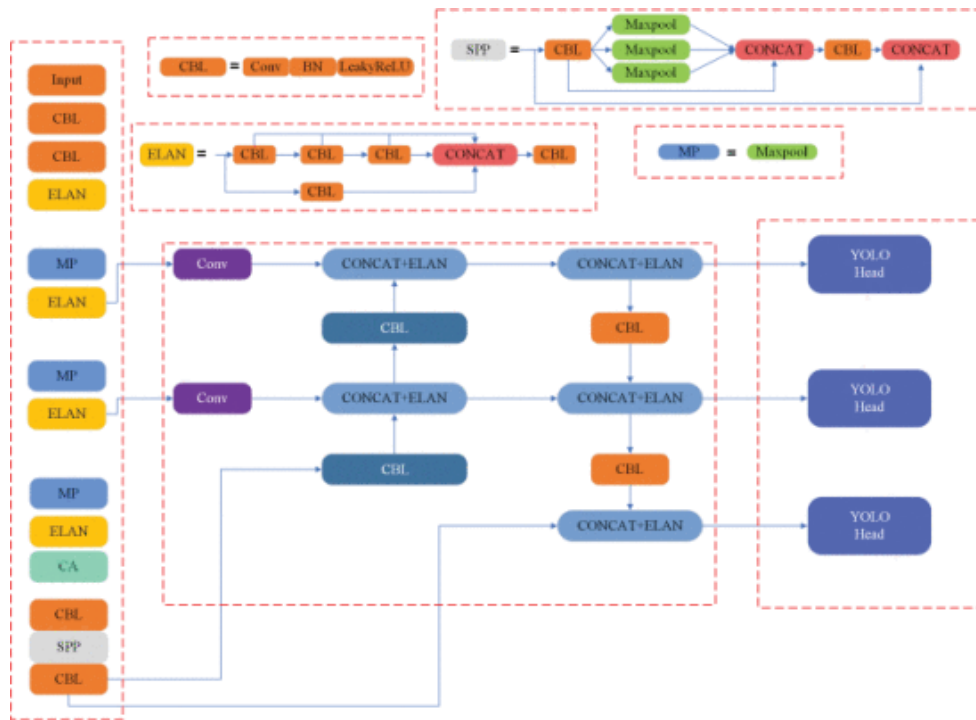


Figura 10: Arquitectura de YOLOv7-veh
Fuente: <https://ieeexplore.ieee.org/document/10828068>

Los vehículos aéreos no tripulados (UAVs) son muy útiles en la tarea de monitorización del tráfico, todo gracias a su capacidad de grabar grandes superficies sin reducir la velocidad del vehículo. En este estudio, Vaishnav V. R. et Al. [13] presentan un proyecto de detección y clasificación de vehículos durante condiciones variadas de luz en entornos de tráfico de la India. El conjunto de datos seleccionado para el entrenamiento y las pruebas, es el dataset **SHIVNATRAJ**⁸, que incluye un espectro muy variado de vehículos comunes en carreteras Indias, agrupando categorías como coches, autobuses, motocicletas, camiones, calesas y LCVs. Se seleccionaron 5 modelos para comparar el rendimiento en términos de detección y velocidad: **YOLOv7**, **YOLOv8**, **Faster R-CNN** y el transformador **DINO-DETR**. Los resultados obtenidos, observables en la tabla 2, muestran que el modelo seleccionado, **YOLOv8** consigue el mejor balance entre precisión y velocidad. **YOLOv7** es inferior en precisión, **DINO-DETR** sufre de una muy baja velocidad de inferencia (8 FPS)

⁷El dataset es accesible por Bitvehicle

⁸El dataset es accesible por <https://www.shivnatraj.com>

y tanto **YOLOv11** como **Faster R-CNN** son modelos muy precisos, pero computacionalmente menos óptimos.

Modelo	FPS	mAP50 (día)	Fortalezas
YOLOv7	20 FPS	79%	Preciso, menos eficiente
YOLOv8	15-18 FPS	81%	Rápido, ligero
YOLOv11	12 FPS	83%	Muy preciso, gran latencia
Faster R-CNN	10 FPS	80%	Preciso, gran latencia
DINO-DETR	8 FPS	84%	Mejor detección de noche

Tabla 2: Resultados de YOLOv8 y YOLOv11 en entornos de tráfico

La reconstrucción del pavimento urbano es esencial para la gestión de la infraestructura de carreteras. Con esto en mente, Han Liu y Ronggui Ma [14] idean un sistema de reconstrucción del pavimento integrando la adquisición de un dataset de imágenes UAV, una **Altitude-Robust Vehicle Detection Network (AR-VDN)** y algoritmos de optimización de la vista adaptada al tráfico. Todas las imágenes fueron obtenidas a través de UAVs a diferentes altitudes y en 4 localizaciones distintas, incluyendo una arteria urbana, una carretera y 2 autopistas; el número total de imágenes después de una limpieza terminó siendo 1885 imágenes restantes en alta calidad. Tras barajar y comparar varios algoritmos de detección, el modelo base seleccionado fue el **YOLOv8n**. Para finalizar siendo la red **AR-VDN**, visible en la figura 11, a este modelo se le aplicaron varias modificaciones:

- Se introduce un módulo de adaptación dinámica de resolución (DRA) para la **variación de escala** de los vehículos que aparecen pequeños a grandes altitudes.
- El backbone de **YOLOv8n**, CSPDarkNet53, se cambió por **HRNet_lite**, que mantiene los mapas de características de alta resolución a través de la fusión de características para **preservar los detalles de grano fino**.
- El módulo de red de agregación de caminos (PAN) de **YOLOv8n** se sustituye por un *Transformer Encoder* de múltiples capas, que segmenta la imagen y usa *embeddings* lineales para **extraer representaciones localizadas**.
- La cabeza de detección se reemplaza por un FCOS, que predice directamente la categoría, coordenadas del centro y el tamaño de la *bounding box*.

AR-VDN consiguió un rendimiento considerable en los datasets: con un mAP50 de **99.32%** (YOLOv8n con 95.52%) y un mAP75 de **94.25%** (YOLOv8n con 63.60%). En cuanto al rendimiento, el modelo tarda **5.1 ms** por imagen o **115 FPS** en una *workstation* con 48GB RAM y una GPU NVIDIA RTX 3080Ti.

Leyre Encio et Al. [3] proponen un sistema de detección de vehículos por la noche, un problema común debido a la falta de visibilidad tanto para los vehículos autónomos como para la gestión del tráfico. Este sistema que se adapta a las condiciones de iluminación, es una red neuronal basada en **CenterNet** (que a su vez se basa en CornerNet) y cuya arquitectura se divide en 2 bloques fundamentales: un backbone de CNNs y un bloque

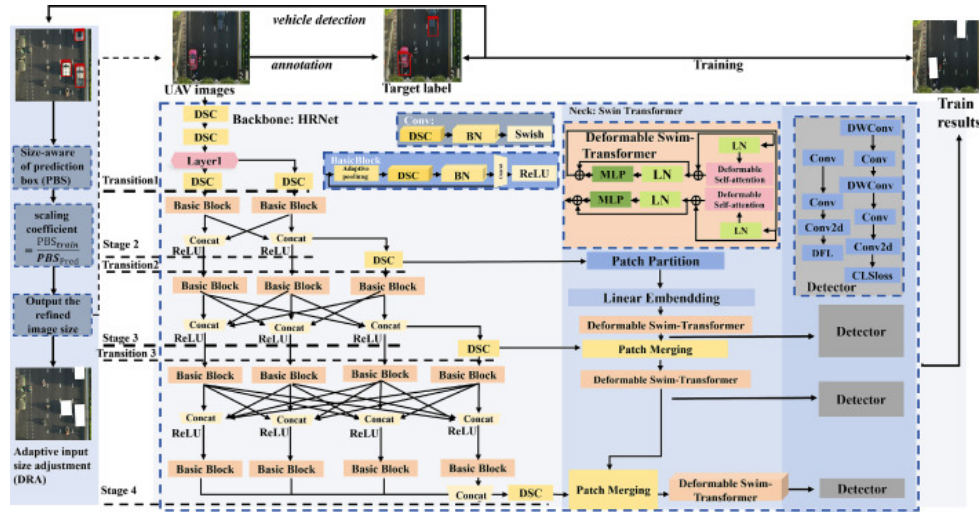


Figura 11: Arquitectura de la red AR-VDN

Fuente: <https://www.sciencedirect.com/science/article/pii/S0926580526001342>

de detección. El backbone se basa en una arquitectura **Hourglass**, mostrada en la figura 12, seleccionada por su capacidad de **capturar detalles pequeños a múltiples escalas**. Esta arquitectura trata de asimilar un diseño de encoder-decoder, donde la imagen pasa a través de una serie de transformaciones hasta que acaba siendo un *embedding* n-dimensional, al cual se le aumenta la resolución y termina con su tamaño original. Este sistema se ha probado en una CPU Intel Core i9-13900K 5.8 GHz y 2 GPUs NVIDIA RTX 4090 de 24 GB. La evaluación se realizó usando tres datasets diferentes: **BDD100K**, **PVDN** y **FNTVD**⁹. Los resultados obtenidos en los dos primeros datasets son los descritos en la tabla 3, y un rendimiento de **45 FPS**.

Métrica	BDD100K	BVDN
mAP	71.34%	66.21%
F1-score	75.54%	73.74%
Precision	77.36%	79.49%
Recall	73.79%	68.76%

Tabla 3: Resultados de la arquitectura con Hourglass en los datasets BDD100K y BVDN

De forma similar al artículo anterior, N.Jathin Reddy et Al. [15] proponen un sistema de detección de vehículos preciso en entornos con baja visibilidad con el objetivo de ayudar a los sistemas inteligentes de transporte. Este problema puede deberse a una multitud de factores como el desenfoque de movimiento o el deslumbramiento por los faros de otros coches. La investigación propone usar la avanzada extracción de características y la arquitectura optimizada de **YOLOv10** para destacar en la precisión de detección, en la eficiencia computacional y en la solidez. Antes de entrenar el modelo, se aplican varias técnicas de preprocesado: ecualización adaptativa del histograma, reducción del ruido y mejora del contraste. Los experimentos producidos demuestran que **YOLOv10** tiene un rendimiento superior en las métricas de mean Average Precision (mAP), velocidad de

⁹Este dataset es accesible a través de <https://www.gti.ssr.upm.es/data/ficosa>

algo de precisión en la detección, tienen una ventaja considerable en términos de inferencia.

Anchor-free : Este tipo de modelos **predicen el centro o los puntos clave de un objeto** y los agrupa para obtener la *bounding box*. El acercamiento por puntos clave supone detectar características clave del objeto para determinar su posición y forma. Entre los modelos se encuentran **CornerNet**, **RepPoints**, **CenterNet**, **ExtremeNet** y **Grid R-CNN**. Algunos modelos clásicos que se basan en el centro del objeto son **YOLOv1**, **FSAF**, **FCOS**, **GA-RPN**, **FoveaBox**, **YOLOX**, **YOLOv8** y **YOLOv9**.

End-to-end : Los anteriores métodos predicen un conjunto de bounding boxes a través de regresión y clasificación. Los detectores *end-to-end* **determinan la posición y clase de un objeto sin aplicar procedimientos adicionales de preprocesado y/o posprocesado**. Aquí se incluyen los modelos **DeFCN**, **Sparse R-CNN** o **DETR**, el cual utiliza una red neuronal basada en transformadores.

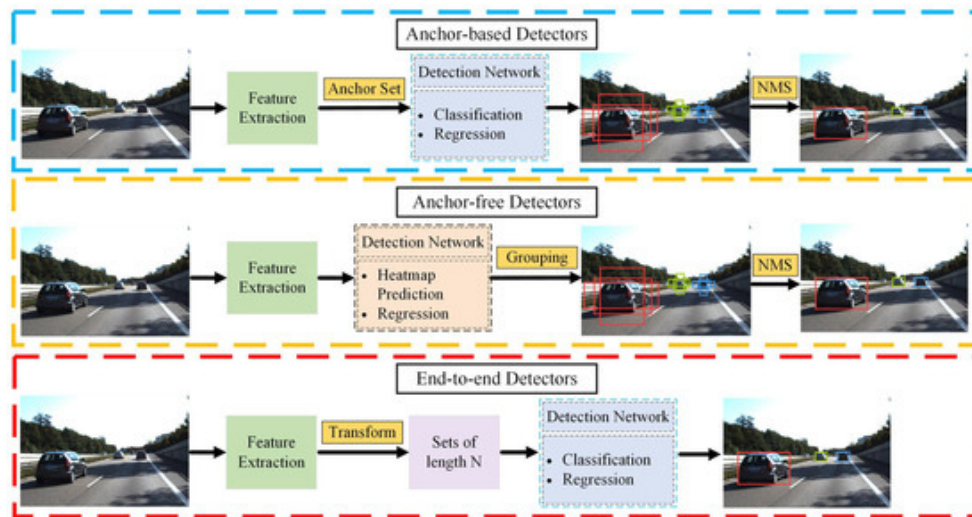


Figura 13: Funcionamiento de distintos métodos de detección de vehículos

Fuente: <https://www.mdpi.com/1424-8220/24/10/3088>

2.2 Detección y lectura de matrículas

Esta tarea no es un problema que se ha podido resolver recientemente; ya hace 8 años, Hui Li et Al. [17] crean un sistema de detección y reconocimiento de matrículas usando redes neuronales profundas (DNNs). Una red neuronal se entrena con imágenes de texto para poder reconocer las letras y números de la matrícula y se entrena una segunda red neuronal para diferenciar entre la matrícula y otras apariciones de caracteres alfanuméricos. La primera CNN consta de 4 capas con la estructura visible en la figura 14, para reconocer 26 letras, 10 números y una clase para no-caracteres.

Para detectar las matrículas, se redimensiona la entrada en 12 escalas y se calcula el mapa de caracteres en cada escala evaluando la segunda CNN con un **método de ventana deslizante** para no perder ningún carácter cerca de los bordes. Las *bounding boxes* can-

didatas se generan en cada escala usando el algoritmo **RLSA**¹⁰, donde píxeles vecinos se conectan si hay cierto espaciado menor que un umbral fijo, y el análisis **CCA** que produce las *bounding boxes* iniciales.

Además, las *bounding boxes* producidas se someten a un proceso de refinamiento para **filtrar las matrículas en términos de limitaciones geométricas** o **redimensionar en casos donde no se haya reconocido la matrícula completa**. Finalmente, se entrena una tercera CNN de 36 clases, con la estructura de la figura 15, para extraer las características de la imagen de la matrícula devuelta por la segunda CNN y una red neuronal recurrente bidireccional (BRNN) con LSTM¹¹ para reorganizar la secuencia de características. Una CTC recibe la salida de la RNN y devuelve la matrícula en los caracteres alfanuméricos.

Layer type	Parameters
Soft-max	37 classes
Fully connected	#neurons: 37
Dropout	Prop: 0.5
Fully connected	#neurons: 512
Maxpooling	p: 2×2, s: 2
Convolution	#filters: 384, k: 2×2, s:1, p:0
Maxpooling	p: 4×4, s: 4
Convolution	#filters: 120, k: 5×5, s:1, p:0
Input	24×24 pixels gray-scale image

Figura 14: Estructura de la primera CNN

Fuente: <https://www.sciencedirect.com/science/article/pii/S0262885618300155>

Este proceso se ha podido probar usando una NVIDIA Tesla K40c GPU con 6GB de VRAM con un entrenamiento donde la primera CNN se entrena con 138,000 imágenes de caracteres y 900,000 imágenes de no-caracteres, la segunda con recortes de 3,000 imágenes de matrículas y la tercera con las imágenes de no-caracteres. Este proyecto alcanza, en la parte de detección, una precisión del **97.56%** y un *recall* del **95.24%** en el dataset Caltech¹². En la parte de reconocimiento se usa el dataset AOLP¹³, en el que se alcanza un **94.85%** de precisión.

Hay una multitud de métodos de Deep Learning que sirven para reconocer matrículas. Laixiang Xu et Al. [18] presentan un repaso sobre varias formas de atajar esta tarea. El primer paso es la presentación de varios conjuntos de datos (por resumir, sólo se presentan algunos variados, no los mejores para este trabajo):

- **OCR Dataset**¹⁴: Contiene 3.8M+ de imágenes de matrículas etiquetadas.

¹⁰Método de segmentación de bloques y discriminación de texto: PythonRLSA

¹¹Arquitectura que permite que las RNNs puedan aprender dependencias a largo plazo en secuencias de datos.

¹²El dataset es accesible por: <https://data.caltech.edu/records/fmbpr-ezq86>

¹³El dataset es accesible por: <https://github.com/AvLab-CV/AOLP>

¹⁴El dataset es accesible por: OCRPlates

Layer type	Parameters
Soft-max	36 classes
Fully connected	#neurons: 36
Dropout	Prop: 0.5
Fully connected	#neurons: 1000
Dropout	Prop: 0.5
Fully connected	#neurons: 1000
Maxpooling	p: 3×3, s: 1
Convolution	#filters: 512, k: 3×3, s:1, p:1
Convolution	#filters: 512, k: 3×3, s:1, p:1
Maxpooling	p: 3×3, s: 2
Convolution	#filters: 256, k: 3×3, s:1, p:1
Convolution	#filters: 256, k: 3×3, s:1, p:1
Maxpooling	p: 3×3, s: 2
Convolution	#filters: 128, k: 3×3, s:1, p:1
Maxpooling	p: 3×3, s: 1
Convolution	#filters: 64, k: 3×3, s:1, p:1
Input	24×24 pixels gray-scale image

Figura 15: Estructura de la tercera CNN

Fuente: <https://www.sciencedirect.com/science/article/pii/S0262885618300155>

- **License Plate Digits Classification Dataset**¹⁵: Está especializado en el reconocimiento de los dígitos de las matrículas.
- **PKU**¹⁶: 10 millones de imágenes de vehículos con varias localizaciones, iluminación y condiciones atmosféricas.
- **RodoSol-ALPR**¹⁷: 20000 imágenes de 1.280 x 720 píxeles, capturadas de día y noche y en días claros y lluviosos en distintos peajes de Brasil.

Los algoritmos descritos se pueden dividir en los tipos de detectores ya descritos en el último artículo del estado del arte de detección de vehículos:

- **Target framework:** Los métodos tradicionales para la detección, como HOG+SVM y Haar Cascade, se dejan atrás para usar algoritmos de Deep Learning como R-CNN o YOLO. Este tipo de detección se pueden dividir en dos tipos:
 - **Two-stage:** Primero se genera las regiones propuestas, se clasifican y se aplica regresión a la bounding box. Este es el caso de Faster R-CNN, la cual genera y selecciona las cajas candidatas a través de una red de sugerencias de región (RPN). Segundo, alinea las características y realiza un reconocimiento preciso a

¹⁵el dataset es accesible por: LPDigits

¹⁶el dataset es accesible por: <https://github.com/PKU-IMRE/PKU-Vehicles>

¹⁷el dataset es accesible por: <https://github.com/raysonlaroca/rodoSol-alpr-dataset>

través de ROI Pooling. Un ejemplo se puede ver sobre la figura 16.

- **One-stage:** Este método hace las dos cosas conjuntamente: regresión de la probabilidad de clase del objeto y de las coordenadas de la bounding box en cada posición espacial del mapa de características. Algunos ejemplos son la familia YOLO, SSD y sus derivados. Un ejemplo se puede ver sobre la figura 17.
- **Anchorless frame:** El ejemplo más claro es con el modelo CenterNet, que se basa en la estimación de los puntos clave. Este hereda la arquitectura de CornetNet, reconstruyendo el objetivo de la detección a partir de los pares de esquina de la bounding box hasta la estimación conjunta de escala central.

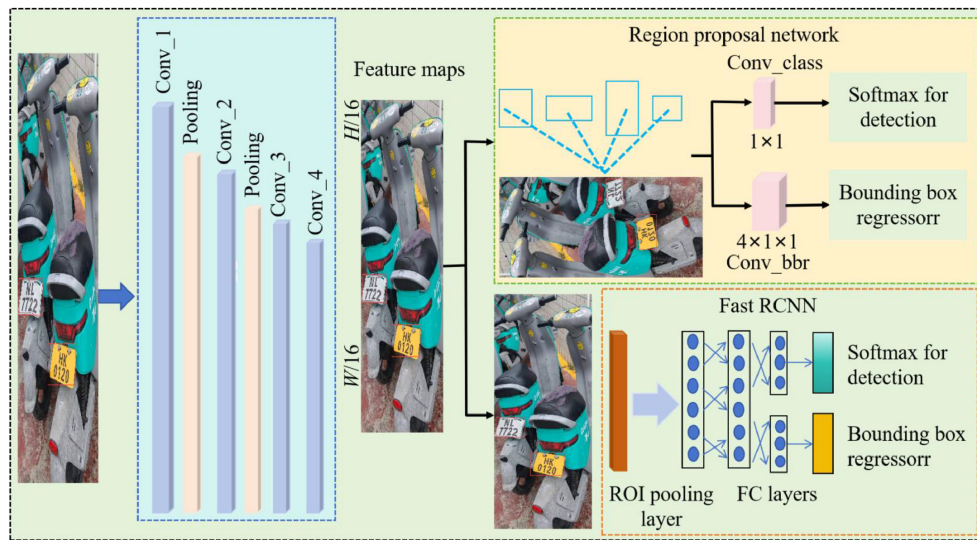


Figura 16: Flujo de un detector de dos etapas en reconocimiento de matrículas

Fuente: <https://www.sciencedirect.com/science/article/pii/S0893608026003047>

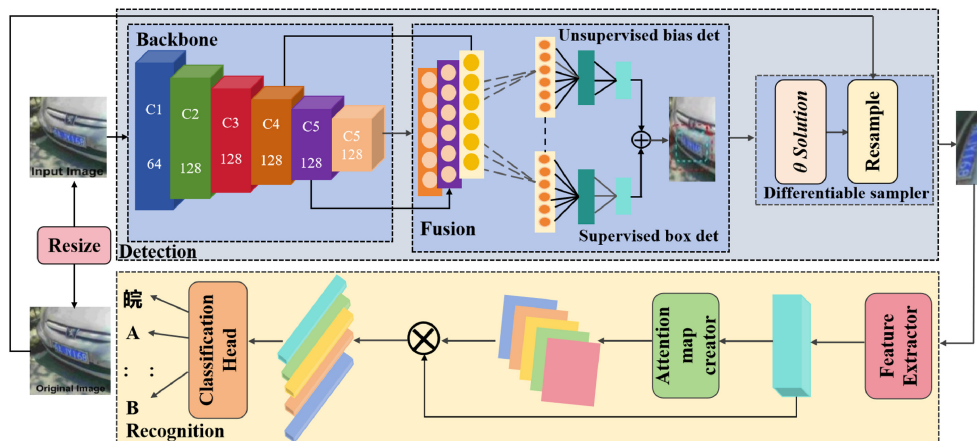


Figura 17: Flujo de un detector de una etapa en reconocimiento de matrículas

Fuente: <https://www.sciencedirect.com/science/article/pii/S0893608026003047>

Se han probado diversos métodos con muy buenos resultados tanto de detección como de rendimiento en cuestión de FPS. Aunque aparecen (y se han probado) muchos más métodos en el trabajo, en la tabla 4, sólo se dispondrán tres de ellos por cada método de

detección (en orden: one-stage, two-stage y anchorless).

Modelo	Dataset	N° de imágenes	Accuracy	FPS	Params(M)	FLOPs
FCOS-SCCA	CCPD	265,500	95.8%	38.6	2.5	5
YOLOv8n	Mobile phone	5,546	91.4%	58	2.5	11.2
MobileNet	Mobile	82,067	95.76%	60	3.4	3
VGG16	CCPD	280,000	91.5%	32	138	15.5
RNN	RFID-Station	1M	85.19%	20	2.1	4.5
Mask-RCNN	Camera	7,712	97%	5	40	200
MobileNetV2	LFW	63,236	99.46%	60	3.4	3
Transformer	CCPD	2,600	88.64%	12	10.2	18.5
DGNet	ELPR	9,342	94.69%	10	6.8	9.5

Tabla 4: Resultados de diferentes arquitecturas en la tarea de detección de matrículas de vehículos

En cuestión de reconocimiento de matrículas, la tecnología ha avanzado mucho, permitiendo capturar, procesar, analizar e interpretar las imágenes de los vehículos para identificar correctamente los caracteres de la matrícula del vehículo, como se puede apreciar en la figura 18, con una estructura de un modelo de Deep Learning para esta tarea. Como en el caso anterior, se han probado muchos modelos en el trabajo; en la tabla 5 se disponen solamente 9 de ellos.

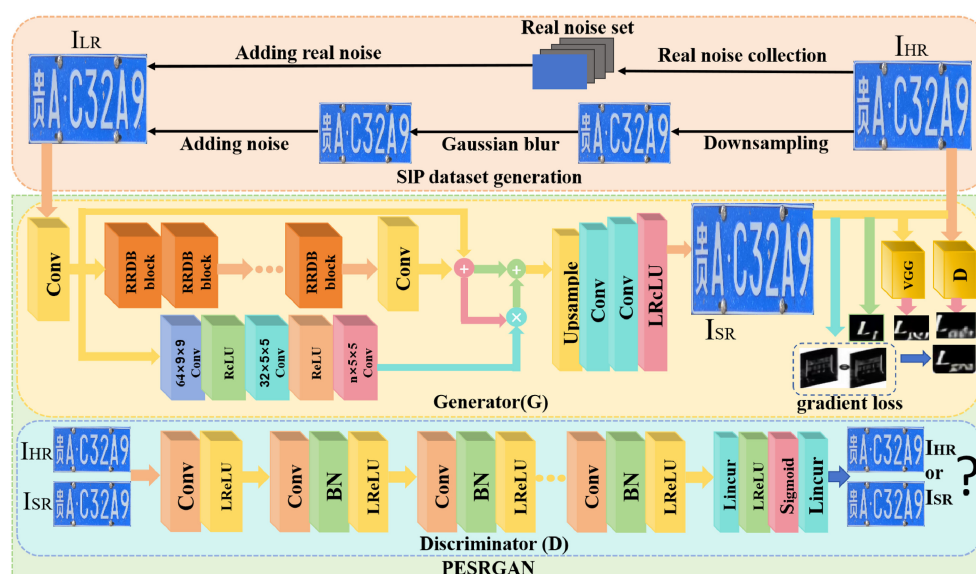


Figura 18: Estructura de un modelo de Deep Learning para el reconocimiento de matrículas

Fuente: <https://www.sciencedirect.com/science/article/pii/S0893608026003047>

La detección y reconocimiento de matrículas es una tarea muy importante cuando se quiere la automatización de servicios aduaneros como el peso de camiones. Este también es el caso de la India, cuyas matrículas comprenden una gran variedad de tamaños, fuentes o formas, por lo que el desarrollo de un sistema automático de reconocimiento de matrículas junto al despliegue de un conjunto de datos es necesario para carreteras para este país. Sai Sirisha Nadiminti et Al. [19] lanzan un dataset de 1,500 imágenes, escalable y reproducible hacia otras implementaciones para esta misma tarea. El conjunto de datos se ha generado

Modelo	Dataset	N° de imágenes	Accuracy	FPS	Params(M)	FLOPs
Hourglass	UFPR-ALPR	3,849	96.19%	110	10	4
YOLOv8	AOLP	5,546	99.02%	101	52	525
DEN-YOLO net	Tunisia	265,000	98.9%	45	3.5	7.2
AENet	CLPR	280,000	98.4%	58	2.5	11.2
CNN RNN	CBLP	30,000+	98.5%	61	3.15	0.57
YOLOv5	UC3M-LP	4,158	98.8%	190	7.3	8.0
CenterNet	OpenALPR	1,729	92.3%	74.9	3.4	2.2
LPRNet	CCPD	36,715	96.00%	180	4.2	1.1
YOLOv11	CSLPD	3,796	93.30%	18	2.6	3.5

Tabla 5: Resultados de diferentes arquitecturas en la tarea de reconocimiento de matrículas de vehículos

a partir de imágenes de páginas como OLX y fotografiando vehículos en carreteras, y etiquetadas con la herramienta labelImg siguiendo el formato VOC Pascal. El algoritmo que se crea es RpNet, que está comprendido por dos módulos:

- **Módulo de detección wR2:** CNN profunda con 10 capas convolucionales para **extraer mapas de características a diferentes escalas**.
- **Módulo de reconocimiento fh02:** Explota capas de agrupación de regiones de interés para **extraer mapas de características de interés** y varios clasificadores para **predecir el número de la matrícula** en la imagen introducida. El primer clasificador se encarga de predecir el código de territorio, que son 2 caracteres, y los 8 siguientes son caracteres alfanuméricos.

El rendimiento conseguido se encuentra en un **99.3%** de precisión considerando un IoU de 0.7 en las imágenes del subconjunto de **CCPD: ccpd base** y un **84.1%** en el subconjunto **ccpd.weather**. El reconocimiento se considera válido si el IoU es mayor que 0.6 y todos los caracteres se han reconocido correctamente; se ha conseguido un **98.5%** de precisión en las imágenes del subconjunto **ccpd base** y un **85.1%** en el subconjunto **ccpd challenge** con una tiempo de inferencia de 61 FPS. El hardware utilizado para el entrenamiento del modelo es un servidor GPU con 8 CPUs Intel Xeon E5-2682 v4 @ 2.50GHz, 60GB RAM y una Nvidia GPU Tesla P100 PCIe 16GB. La realización de las pruebas se realizaron en portátiles con 8 3.40 GHz Intel Core i7-6700 CPU, 24GB RAM y una Quadro M4000 GPU.

No siempre se tiene la potencia que se requiere para desplegar un programa que necesita gran cantidad de recursos: a veces es necesario crear un programa que use menos recursos. Natalya Kharina et Al. [20] diseñan un programa que localiza, reconoce y segmentar matrículas en una entrada a un área restringida. El proceso, explicado con imágenes en la figura 19 se divide en diferentes pasos:

- **Preprocesamiento:** Mejora del contraste en la imagen en escala de grises y se binariza con un umbral dado por el método Otsu¹⁸.

¹⁸Método de segmentación que sirve para binarizar imágenes de forma que se escoge el umbral que proporcione la mejor separación entre dos clases diferentes de píxeles en términos de sus niveles de intensidad.

- **Localización de la matrícula:** Se buscan contornos en la imagen binaria, limitándose a contornos de entre 3.5 a 5 píxeles de rango, y el fragmento localizado (la supuesta matrícula detectada) se revisa para comparar su similitud con una matrícula real a partir de las siguientes características:
 - Se calcula otro umbral para **obtener otra imagen binaria adicional** a partir de un fragmento de semitonos.
 - **Se calcula el coeficiente de cobertura de los píxeles negros** a partir de todos los píxeles de la imagen de la matrícula. Si este coeficiente no se incluye en los intervalos (0.1, 0.3] o (0.6, 0.75), este fragmento de imagen no se incluye para reconocer la matrícula.
- **Rotación y normalización:** La rotación aplica un **filtro lineal** para suavizar la distorsión de la imagen y **se recorta** para prevenir información innecesaria de meterse en el fragmento de la matrícula.
- **Segmentación:** La imagen de la matrícula **se binariza usando un umbral dinámico** en el algoritmo Otsu y se calcula el número de píxeles negros en cada columna. Se busca el final del carácter a lo largo del eje X para determinar el intervalo de espacio del símbolo: si es suficientemente largo, se considera como posible posición de un símbolo. Se aplica el mismo cálculo para averiguar la posición en el eje Y, calculando el índice de píxeles negros entre 2 columnas encontradas por la parte anterior. Finalmente, se recorta la imagen. Para prevenir más falsos positivos, se revisa la proporción altura y longitud del recorte: si la longitud no es suficientemente grande, se busca el contorno más cercano al contorno izquierdo del lado derecho.
- **Reconocimiento de texto:** Escalado de los símbolos para tener tamaños similares en los caracteres y selección de los símbolos de acuerdo al coeficiente de correlación más alto entre el fragmento original y el patrón.

El conjunto de datos comprende 2000 imágenes de vehículos con sus matrículas visibles, obteniendo una probabilidad de reconocimiento correcto del **96%**, donde la probabilidad de error en el reconocimiento es del 0.4%, probabilidad de no reconocer del 3.5% y un 1.5% de falsos reconocimientos. El tiempo de procesamiento de una imagen se encuentra en torno a unos **35 ms** en un procesador AMD Ryzen 2600x.

Los sistemas de reconocimiento de matrículas se pueden adaptar a un amplio rango de características; como Hasan Aljzaere et Al. [21], que desarrollan un sistema de reconocimiento en carreteras utilizando las tecnologías de **YOLOv8** y **EasyOCR**. Para probar este sistema, se ha extraído un dataset de 4000 imágenes en blanco y negro de vehículos alemanes de la página web **Roboflow**. El sistema empieza por detectar los vehículos que aparecen en los vídeos, con el uso del algoritmo SORT (*Simple Online and Real-time Tracking Algorithm*) manteniendo la asociación entre el vehículo y la matrícula. Después de seguir al vehículo, se usará el sistema YOLO para detectar las matrículas, y, si la confianza en la detección supera el 85%, triplicando su tamaño para pasarlo al algoritmo EasyOCR.



Figura 19: Resultados del algoritmo de reconocimiento de matrículas

Fuente: <https://ieeexplore.ieee.org/document/9790745>

La estructura del extractor de características del modelo YOLOv8 se divide en 3 partes fundamentales: CSB, C2f y SPPF. El módulo **CSB** con convoluciones 2D y la función de activación SiLU **ayuda en la extracción de características**, **C2f mejora el flujo del gradiente** y SPPF se encarga de aprender las características. A esta estructura le sigue el cuello **PAN-FPN, encargado de** utilizar semántica de alto nivel y la información de la ubicación para **mejorar el uso de características**. La última parte es la encargada de separar las tareas de clasificación y detección.

La tecnología EasyOCR es la encargada de transformar el texto de la imagen en una cadena de caracteres a través de librería de OpenCV, **comparando la imagen recortada con una plantilla guardada en una base de datos** (similar a la funcionalidad Template Matching), detectando texto en más de 80 lenguajes diferentes, en escenarios donde la orientación de caracteres puede ser compleja. Un ejemplo se puede ver sobre la figura 20.

Este sistema da como resultados una precisión del **95%** para la detección de matrículas y un índice de reconocimiento del **94%** usando una confianza en el algoritmo EasyOCR del 85%. El entrenamiento ha consistido de 1000 iteraciones



Figura 20: Ejemplo de reconocimiento de matrícula usando el algoritmo EasyOcr

Fuente: <https://ieeexplore.ieee.org/document/10804754>

Para la detección y reconocimiento de matrículas en entornos restringidos y vehículos que rompen las leyes de tráfico, Debajit Sarma et Al. [22] idean un sistema de tres partes fundamentales:

- **Detección de la matrícula:** Se convierte la imagen de RGB a binaria (más eficiente computacionalmente), mejorando y suavizando la imagen con el uso de *Gaussian Blurring* para eliminar el ruido; a esto le sigue la aplicación del umbral adaptativo (*Adaptive Thresholding*).
- **Segmentación de caracteres:** La imagen 21 detalla los pasos involucrados en esta etapa; fundamental para la siguiente etapa ya que si no se pueden segmentar los caracteres, tampoco se pueden reconocer.
- **Reconocimiento de caracteres:** Un framework de tensorflow junto con una CNN se encargan de reconocer las 36 clases (A-Z y 0-9). La CNN está compuesta de **3 capas convolucionales** con kernels de 5x5 y **2 capas totalmente conectadas**; de las cuales, una de ellas con un *dropout* de 0.5 para prevenir el *overfitting*. La red termina con una capa de *softmax* para dar las probabilidades por clase.

Para probar el sistema propuesto se han escogido dos conjuntos de datos: el primero¹⁹ consta de 443 imágenes de vehículos con diferentes condiciones de visibilidad y ángulos con la matrícula visible. El segundo contiene un subconjunto recortado de 1980 imágenes de caracteres y letras [23] (3410 en total) resultando en 55 imágenes por carácter con 45/10 para los conjuntos de entrenamiento y validación. Las pruebas indican un 95.75% de precisión en el conjunto de caracteres.

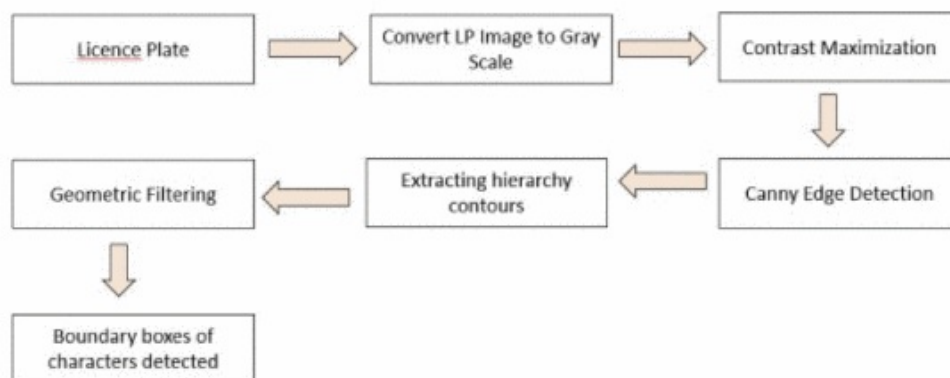


Figura 21: Proceso de flujo de segmentación de matrículas

Fuente: <https://ieeexplore.ieee.org/document/10183638>

Phayung Meesad et Al. [24] combinan el algoritmo **YOLOv10** con un motor **Tesseract OCR** especialmente personalizado para la tarea de reconocimiento automatizado de matrículas (ALPR), fundamental para la gestión del tráfico, proteger la seguridad pública, peajes y el acceso a lugares restringidos. La combinación de estos dos algoritmos tiene como objetivo reconocer matrículas de vehículos con una mezcla de **caracteres tailandeses**

¹⁹El conjunto de datos es descargable de la página CarPlateDetection

y **latinos**, un problema complicado no resuelto en la literatura. El sistema propuesto toma como entrada imágenes o vídeo en tiempo real de cámaras de vigilancia y se somete a varias **funciones de preprocesamiento**: reducción del ruido, ajuste del contraste y técnicas de segmentación, que tienen mucha eficacia a la hora de asolar regiones de las matrículas.

La parte de detección utiliza **modelos YOLO** avanzados ("avanzados" en su momento como YOLOv8, YOLOv9 o YOLOv10) para detectar las matrículas de las imágenes pre-procesadas. Después de que la detección resulte correcta, un motor **Tesseract OCR**, adaptado a matrículas Tailandesas que presentan dos tipos de escritura y fuentes variables para mejorar el reconocimiento y solucionar los problemas lingüísticos inherentes en el conjunto de datos. El modelo **YOLOv10** destaca entre los modelos comparados en este estudio, característico gracias a sus **kernels dinámicos convolucionales (DCK)**, que mejora el proceso de extracción de características a costa de un mínimo aumento de la carga computacional.

El conjunto de datos consta de 50,000 imágenes y 10,000 trozos de vídeo (con una división de 80:20 para los conjuntos de entrenamiento y pruebas) en diferentes condiciones atmosféricas y de iluminación y con cambios notables en las matrículas con colores o fuentes, por ejemplo. Algunos ejemplos se pueden observar en la figura 22. Las pruebas realizadas se llevaron a cabo en una NVIDIA A100 GPU de Google Colab comparando los detectores YOLOv5, YOLOv8, YOLOv9, YOLOv10, Faster R-CNN y SSD, cuyos resultados están descritos en la tabla 6 y muestran la superioridad de YOLOv10 frente a los anteriores modelos YOLO. El reconocimiento también tuvo resultados satisfactorios: el motor Tesseract OCR alcanzó una tasa de reconocimiento del 99.16% y un F1-score del 99.2%.



Figura 22: Ejemplos de matrículas tailandesas
Fuente: <https://www.nature.com/articles/s41598-025-24967-9>

La gestión del aparcamiento urbano es un problema creciente debido a la limitación del espacio, con los métodos tradicionales fallando en los momentos con mayor tráfico y resultando en ineficiencias, uso inautorizado y pérdidas de dinero. El sistema propuesto por Gulmini Pradhan et Al. [25] considera una cámara con una resolución mínima de 1080 con enfoque automático y HDR visualizando múltiples plazas de aparcamiento

Modelo	Accuracy	Precision	Recall	F1-score	Inference time
YOLOv5	96.97%	97.90%	97.10%	97.5%	1.4 ms
YOLOv8	97.79%	98.20%	97.30%	97.7%	1.4 ms
YOLOv9	98.96%	99.8%	97.80%	98.8%	1.4 ms
YOLOv10	99.16%	99.8%	98.60%	99.2%	1 ms
Faster R-CNN	92%	92.50%	92%	92.2%	4.5 ms
SSD	90%	91%	90.5%	90.7%	2.5 ms

Tabla 6: Resultados de diferentes arquitecturas en el reconocimiento de matrículas tailandesas

(incluyendo visión clara de las matrículas). Cuando el vehículo entra a la plaza, es detectado por un **sensor IR** que activa la **cámara**, la cual captura la matrícula. Después de que un modelo **YOLO** detecte el vehículo y compare su posición con la de la plaza, se verifica la información y se procede a capturar la matrícula y empezar el contador. Después de la verificación, la matrícula se captura usando la tecnología **OCR** y vinculándola con la base de datos de los vehículos.

Se aplica *Gaussian Blurring* y *thresholding* (aplicación de un umbral para convertir una imagen en binario) tanto en la imagen del vehículo como en la de la matrícula para reducir el ruido y segmentar la imagen, seguido de un contorneado para localizar y dibujar una bounding box alrededor de una matrícula. Gracias a la aplicación de estas técnicas en la imagen de la matrícula, se puede extraer el texto mejorado con la tecnología Tesseract OCR.

Tesseract OCR es un motor óptico de reconocimiento de texto desarrollado por Google que analiza las características estructurales del texto, resultando en el reconocimiento de patrones en la imagen y transformándola en texto legible para la computadora. Énfasis en los sofisticados algoritmos de reconocimiento que disciernen los caracteres del resto de los elementos visuales irrelevantes en la imagen. Su versatilidad y fiabilidad hacen que sea una herramienta eficaz para una multitud de tareas e industrias que involucren el reconocimiento de texto.

El sistema es eficaz tanto en los entornos con buena visibilidad como con baja, tal y como se puede apreciar en la figura 23, manteniendo una precisión en torno al **95%** en condiciones de día y un **90%** en condiciones de noche.



Figura 23: Resultado de Tesseract OCR en varias matrículas

Fuente: <https://www.nature.com/articles/s41598-025-86441-w>

Hanae Moussaoui et Al. [26] proponen un nuevo sistema de detección y reconocimiento

de matrículas usando el modelo **YOLOv8**, técnicas de procesamiento de imágenes y una técnica OCR para el reconocimiento de texto; el sistema completo se puede observar sobre la figura 24. El conjunto de datos se crea a partir de **270** imágenes de Internet y **CVAT** (Computer Vision Annotation Tool), un programa de código abierto para etiquetar imágenes y vídeos. Estas imágenes contienen vehículos visibles desde distintos ángulos y varias condiciones de iluminación y se dividen en los subconjuntos de entrenamiento, validación y pruebas.

La matrícula detectada por el modelo **YOLOv8** se redimensiona y mejora usando técnicas de preprocesamiento, como *k-means clustering*, *thresholding* y operaciones morfológicas. Este paso es crucial para reconocer los caracteres del texto, sobre todo con el ruido producido por la detección de bordes. De entre los modelos YOLO, se escoge la última versión de YOLO (hasta el momento de hacer el artículo), **YOLOv8**, debido a las mejoras en eficiencia y rendimiento como la introducción de **NMS (Non-Maximum Suppression)**, que redefine las bounding boxes con baja confianza o duplicadas. Algunos ejemplos de detección sobre varias matrículas se pueden observar en la figura 25. De entre todas las técnicas **OCR**, se ha seleccionado **EasyOCR**, debido a su rápido procesamiento en tiempo real y simple uso en muchos programas en Python.

Los resultados logran alcanzar una precisión del **74%** en 40 iteraciones y un **96.37%** al cabo de 68 iteraciones.

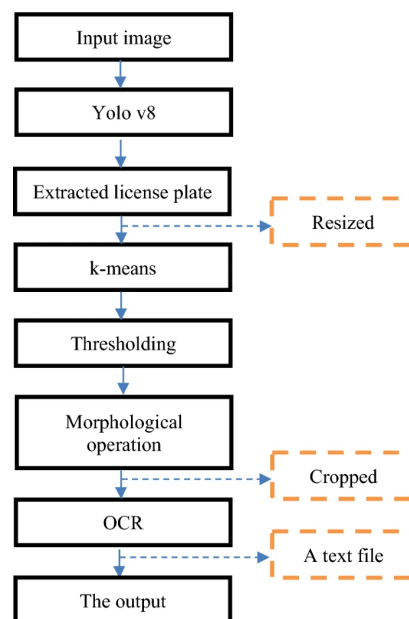


Figura 24: Flujo del sistema de reconocimiento de matrículas con YOLO, técnicas de preprocesamiento y OCR

Fuente: <https://www.nature.com/articles/s41598-025-86441-w>

La expansión incesante de las ciudades se ha visto beneficiada por mejoras en cuestiones como la ciencia y la tecnología, lo que resulta en un crecimiento del número de vehículos en los núcleos de población, incluyendo Corea del Sur. Esto da pie al problema descontrolado del acceso a un aparcamiento, incluyendo espacios para gente con

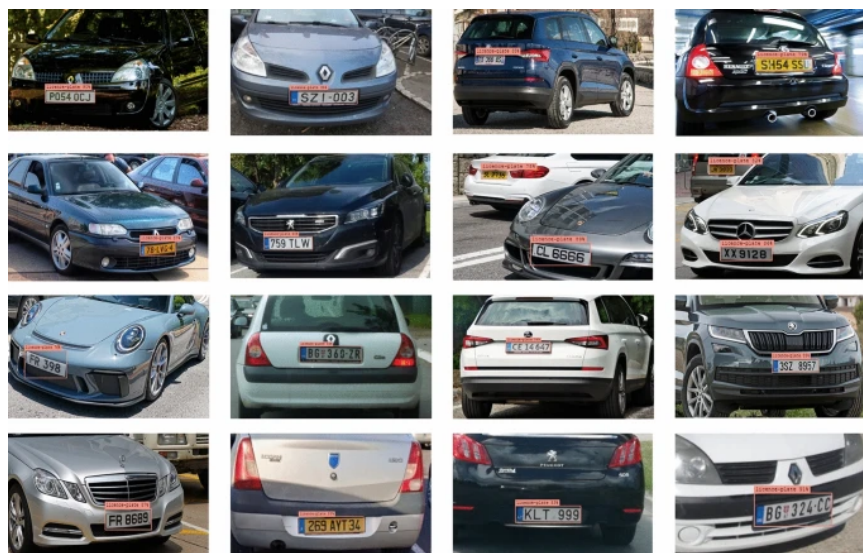


Figura 25: Ejemplos de detección de matrículas usando el modelo YOLOv8

Fuente: <https://www.nature.com/articles/s41598-025-86441-w>

discapacidad (plazas para minusválidos). Para abordar este problema, Dhiraj Neupane et Al. [27] idean *Shine*: un sistema de detección novedoso del vehículo, la matrícula (incluido el reconocimiento de los caracteres), y las pegatinas o marcas de vehículos para deshabilitados. La metodología completa que se ha llevado a cabo en este trabajo se puede dividir en:

- **Recolección de datos:** 30,000 imágenes de vehículos coreanos y de placas de acceso fueron recogidas de plazas de aparcamiento, páginas del gobierno y conjuntos de datos online. Estas marcas se pueden encontrar en cualquier ángulo por lo que se ha aplicado *Data Augmentation* con rotaciones de hasta 60 grados con intervalos de 30.
- **Etiquetado:** Unas 29,000 imágenes se anotaron con el formato YOLO y el formato Pascal VOC para detección de objetos con la herramienta **makesense.ai**. Las imágenes ya anotadas se dividieron en conjuntos de entrenamiento y validación con una proporción de 90/10. El número de clases en el conjunto es de 69 incluyendo el vehículo, el tipo de matrícula (4 tipos), caracteres alfanuméricos y las marcas, como se puede observar en la figura 26.
- **Entrenamiento y pruebas:** Específicamente se entrenaron los modelos YOLOv4, YOLOv5s, YOLOv7 y Faster R-CNN a través de *Transfer Learning*, permitiendo usar los modelos preentrenados para adaptarse a esta tarea (o a cualquier otra). Los resultados dispuestos en la tabla 7 demuestran la superioridad de YOLOv7 en cuestión de precisión de detección frente a modelos más antiguos YOLO y a Faster R-CNN; sin embargo YOLOv5 supera a todos los demás en cuestión de inferencia por imagen. Finalmente, se termina eligiendo el modelo YOLOv7 debido a la prioridad por la precisión y una velocidad de inferencia aceptable para el proyecto.
- **Post-procesado:** Después de aplicar la detección con YOLOv7, el resultado es proce-

sado por medio de matemática simple para sacar la matrícula y los números de las marcas, mostrado en la figura 27.

La figura 28 provee toda la información acerca del funcionamiento del sistema Shine, incluyendo los componentes, el trabajo y los algoritmos, donde (a) enseña los contenidos el hardware completo del sistema, (b) la operación a seguir a partir de una imagen y (c) una demostración del post-procesado que se lleva a cabo para verificar los derechos de accesibilidad del vehículo.

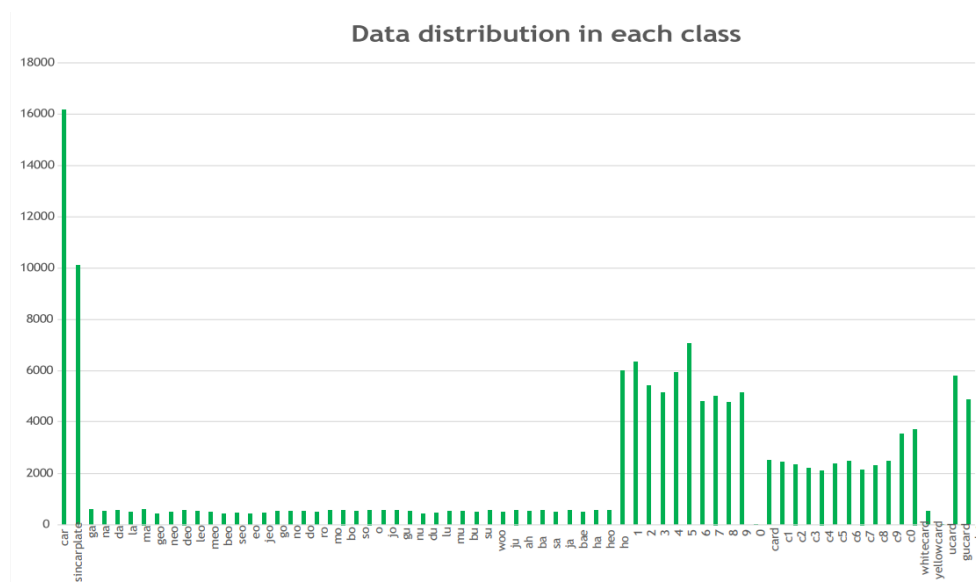


Figura 26: Distribución de clases del conjunto de datos del sistema Shine

Fuente: <https://arxiv.org/pdf/2302.00837>

Modelo	mAP50	mAP50-95	Precision	Recall	Inference time
YOLOv7	92.16%	66.57%	94.7%	90.84%	60ms
YOLOv5s	87.77%	62.13%	94.51%	83.08%	12.63ms
YOLOv4	83.17%		92%	87%	260.1ms
Faster R-CNN	70.50%		69.94%	72.9%	292.33ms

Tabla 7: Resultados de los modelos de detección usados en el sistema Shine

La identificación automática de matrículas de vehículos se pueden usar para comparar dichas matrículas con bases de datos de vehículos robados, sin seguro, criminales,... La tecnología de OCR (*Optical Character Recognition*) es una de las más explotadas para reconocer matrículas y aunque, es muy eficaz, tiene varias limitaciones como el ruido, las condiciones atmosféricas, la luz o la oclusión ambiental. Para reparar este hueco, Nouar AIDahoul et Al. [28] proponen el uso de modelos de lenguaje visual (VLMs) como *OpenAI GPT4o*, *Google Gemini 1.5* o *Anthropic Claude 3.5 Sonnet* en un trabajo comparativo junto con un VLM propio: *VehiclePaliGemma*. Los conjuntos de datos que se van a utilizar son:

- **Complex Plate Datasets:** 258 imágenes de matrículas malayas que están borrosas, difíciles de reconocer para los métodos de OCR y que sólo se utilizarán para evaluar el modelo.

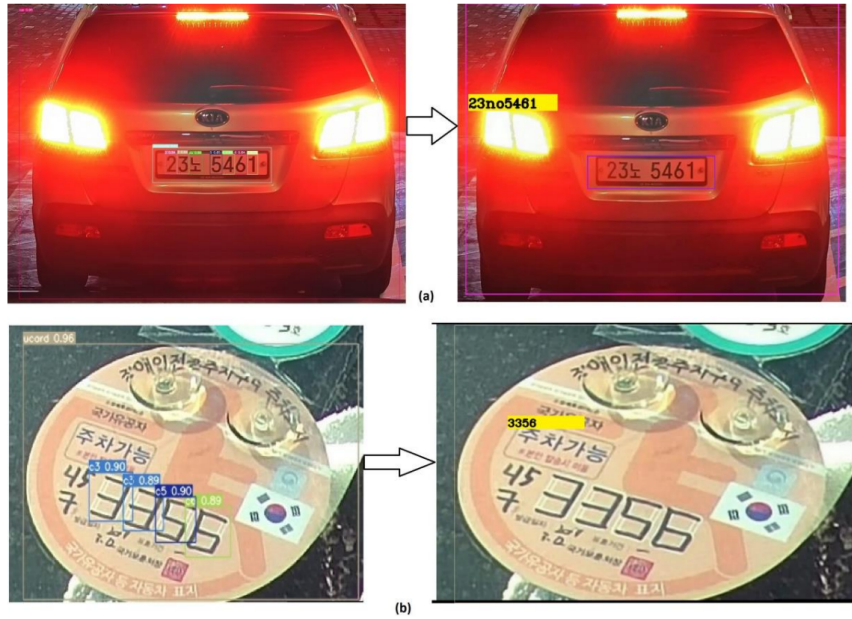


Figura 27: Post-procesado de (a) Matrícula (b) Marca

Fuente: <https://arxiv.org/pdf/2302.00837>

- **Training Plate Datasets:** 600 imágenes sintéticas para hacer fine-tuning al modelo *PaliGemma*, cada una con un tamaño de 120 x 50 píxeles, rotadas en 5 grados, se difuminaron y se sometieron a ruidos Gaussianos y "sal y pimienta".
- **Diverse Car Datasets:** 140 imágenes extraídas de Internet etiquetadas por mayoría en votación 3 modelos distintos

La solución propuesta es una combinación de Inteligencia Artificial de procesamiento de lenguaje y visual de las VLMs para entender los caracteres sin necesidad de preprocesado. El diagrama de la solución se puede observar en la figura 29. Después de incluir en las pruebas de reconocimiento 3 modelos de OCR: EasyOCR, KerasOCR y Tesseract, los resultados observados en la figura 8 indican que estos modelos OCR que mostraban buenos resultados en este tipo de tareas, fallan al intentar reconocer las imágenes del dataset **Malaysian license plates**²⁰. De entre los VLMs preentrenados, GPT-4o destaca reconociendo correctamente 222 de las 258 matrículas, seguido de su versión más pequeña, GPT-4o, y superando ambos con una buena diferencia al resto de VLMs, sean pequeñas o grandes.

Sin embargo, el modelo que destaca es VehiclePaliGemma, identificando 226 matrículas correctamente; una diferencia considerable con su modelo sin hacer *fine-tuning*, que sólo pudo identificar 178. Además el modelo es capaz de predecir la matrícula de un vehículo a una velocidad de 7 FPS en una A100-80GB GPU. Para analizar más en profundidad los resultados es necesario observar la figura 30, donde los modelos fallan más al intentar identificar los caracteres.

Cuanto más claro sea una celda, menos veces ha conseguido el modelo identificar ese

²⁰Tesseract está basado en redes LSTM, EasyOCR usa el algoritmo Character-Region Awareness for Text detection (CRAFT) junto con RCNNs y KerasOCR usa el algoritmo CRAFT usa RCNNs o transformadores.

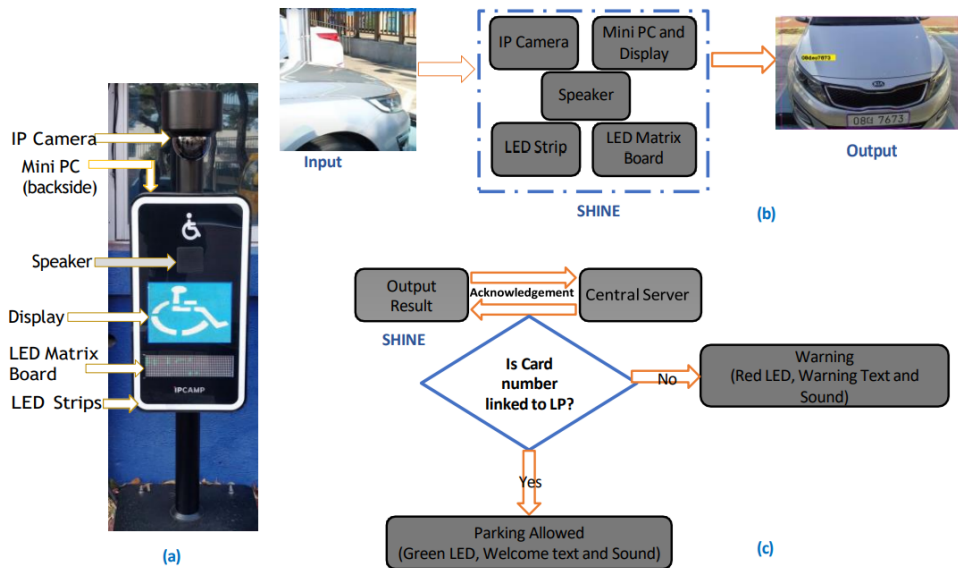


Figura 28: Componentes del sistema Shine

Fuente: <https://arxiv.org/pdf/2302.00837>

caracter correctamente. De aquí se pueden sacar observaciones como que la "I" tiene muchos problemas para ser identificada por algunos modelos, pero por otros es totalmente lo contrario o, la más evidente, que todos los modelos tienen problemas para identificar el caracter "Q".

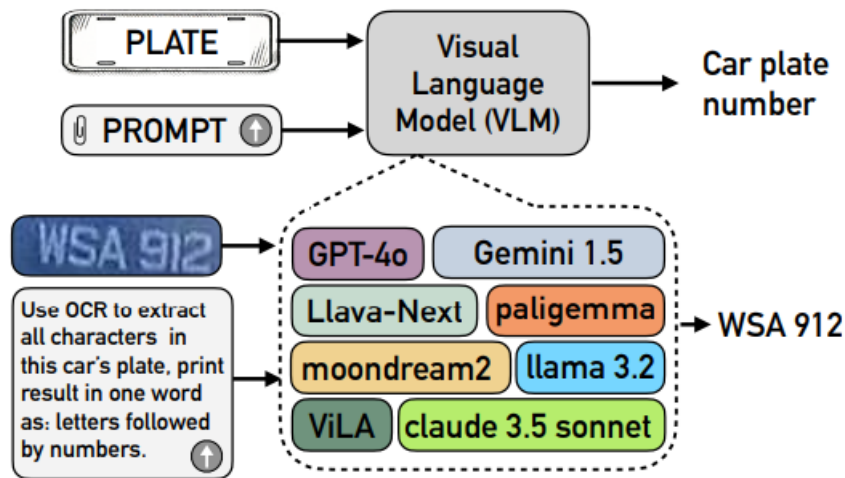


Figura 29: Diagrama de uso de VLMs para reconocer caracteres en matrículas

Fuente: <https://arxiv.org/pdf/2412.14197>

Con la idea de gestionar de manera eficiente el tráfico, la seguridad, el reconocimiento de vehículos y los aparcamientos, Sergey Zherzdev y Alexey Gruzdev [29] crean el primer método en tiempo real de reconocimiento de matrículas que no use RNNs, **LPRNet**. Estudios previos sugerían el uso de los modelos VGG, ResNet o GoogLeNet para esta tarea; en vez de eso, se construye una red neuronal inspirada en los "bloques de fuego" de SqueezeNet y los bloques de Inception. El diseño consta de varias partes:

Modelo	Matrículas correctamente predichos	Precisión (%)
EasyOCR	79	32.95%
Tesseract	97	36.74%
KerasOCR	107	40.53%
moondream2	102	39.5%
VILA	144	55.8%
LLaVA-NeXT-7b	147	57%
Gemini 1.5 flash	200	77.5%
GPT-4o-mini	220	85.7%
LLaVA-NeXT-34b	152	58.9%
Gemini 1.5 Pro	185	71.7%
GPT-4o	222	86%
Llama 3.2 Instruct	175	67.83%
Claude 3.5 Sonnet	186	72.1%
Pre-trained PaliGemma	178	69%
VehiclePaliGemma	226	87.6%

Tabla 8: Resultados de diferentes VLMs en reconocimiento de matrículas

- Preprocesamiento con una red de ubicación (LocNet), visible en la figura 31, con una **capa de transformación espacial** para conseguir mejores características.
- El extractor de características de la figura 32 (donde el bloque básico es el de la figura 33) permite **extraer las características espacialmente distribuidas**. El kernel de la última capa convolucional (1x13) utiliza el contexto local en vez del LSTM de las RNNs.
- Clasificación de los caracteres por cada posición.
- Procedimiento post-filtro, que devuelve las N secuencias más probables encontradas por *Beam Search* y devuelve la que más encaje con unas marcas predefinidas según las regulaciones locales para las matrículas.

El entrenamiento se ha realizado con la herramienta de Tensorflow, usando el optimizador Adam, una tasa de aprendizaje de 0.001 y una escala de ruido del gradiente de 0.001. La tasa de aprendizaje se reduce en un 10% cada 100,000 iteraciones, entrenándose el modelo con 250,000 iteraciones en total. La red de ubicación LocNet se activa a partir de la iteración 5,000, ya que al principio del entrenamiento puede llevar a degradar los resultados al no poder recibir correctamente los gradientes. El dataset seleccionado fue el Chinese License Plates dataset, con 11696 imágenes de matrículas, donde el modelo pudo alcanzar un **95%** tardando 3 ms por cada imagen con el uso de una NVIDIA GTX 1080 o 1.3 ms usando una CPU Intel Core i7-6700K.

En cualquier tarea de detección de objetos, el uso de múltiples puntos de vista puede ayudar a resolver muchos problemas de baja visibilidad. Dat Tran-Anh et Al. [30] idean un método de detección de texto en las matrículas utilizando 3 puntos de vista diferentes para restaurar los componentes de la matrícula basándose en estimaciones de nivel y distancias.

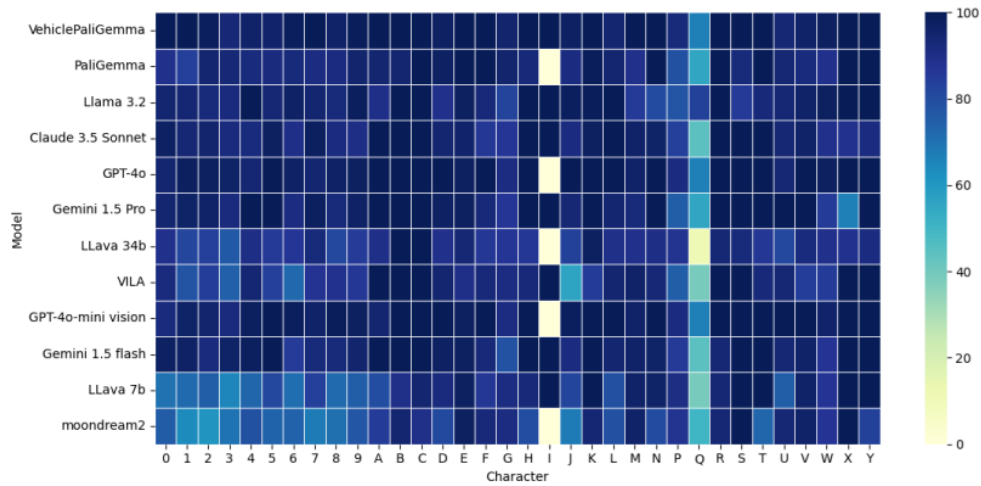


Figura 30: Mapa de calor de la precisión de reconocimiento de caracteres en diferentes VLMs.
Fuente: <https://arxiv.org/pdf/2412.14197>

Layer Type	Parameters	
Input	94x24 pixels RGB image	
AvgPooling	#32 3x3 stride 2	—
Convolution	#32 5x5 stride 3	#32 5x5 stride 5
Concatenation	channel-wise	
Dropout	0.5 ratio	
FC	#32 with TanH activation	
FC	#6 with scaled TanH activation	

Figura 31: Red de ubicación (LocNet)
Fuente: <https://arxiv.org/pdf/1806.10447>

El flujo del reconocimiento de matrículas tiene dos partes indivisibles: la detección de objetos dentro de la imagen (en este caso particular, la propia matrícula, algún elemento substancial del vehículo o, el propio vehículo) y el reconocimiento de los caracteres en la matrícula. Este trabajo consta de 3 partes:

- Detección del vehículo y la matrícula con el modelo YOLOv8, elegido gracias a su alta precisión y velocidad de inferencia. Se recolectaron imágenes de vehículos con las matrículas visibles para crear un dataset propio. Si se detecta la matrícula, el trozo de imagen con esta **se recorta y se envía a la parte 2**.
- Se crea una **nueva función de coste basada en la métrica IoU para solventar predicciones de matrícula de menor tamaño que el real**. Dadas dos imágenes I_1 e I_2 , un modelo DenseNet se entrena para producir una imagen fusionada con la mayor calidad posible. Se computan dos valores w_1 y w_2 como el grado de preservación de la información, a partir de aplicar la función softmax a la división entre la información y el número de clases. La función de pérdida final queda definida como $L = E(w_1 * MSE_{If,I_1} + W_2 * MSE_{If,I_2})$, que se usará para entrenar el modelo de agregación de características: el cual combina características de varios fotogramas en una única representación para el reconocimiento de matrículas.

Layer Type	Parameters
Input	94x24 pixels RGB image
Convolution	#64 3x3 stride 1
MaxPooling	#64 3x3 stride 1
Small basic block	#128 3x3 stride 1
MaxPooling	#64 3x3 stride (2, 1)
Small basic block	#256 3x3 stride 1
Small basic block	#256 3x3 stride 1
MaxPooling	#64 3x3 stride (2, 1)
Dropout	0.5 ratio
Convolution	#256 4x1 stride 1
Dropout	0.5 ratio
Convolution	# class_number 1x13 stride 1

Figura 32: Extractor de características del modelo LPRNet
Fuente: <https://arxiv.org/pdf/1806.10447>

Layer Type	Parameters/Dimensions
Input	$C_{in} \times H \times W$ feature map
Convolution	# $C_{out}/4$ 1x1 stride 1
Convolution	# $C_{out}/4$ 3x1 strideh=1, padh=1
Convolution	# $C_{out}/4$ 1x3 stridew=1, padw=1
Convolution	# C_{out} 1x1 stride 1
Output	$C_{out} \times H \times W$ feature map

Figura 33: Bloque básico del extractor de características del modelo LPRNet
Fuente: <https://arxiv.org/pdf/1806.10447>

- La arquitectura **CnOCR**, cuya arquitectura se puede observar en la figura 34, tiene una inferencia por imagen de solo 0.03 segundos. Se empieza con **múltiples capas convolucionales y capas de agregación** a los que le sigue una capa de **LSTM bidireccional de 100 nodos** que se encarga de capturar la información contextual y las dependencias entre caracteres. La última capa es una densa con activación *softmax* de 53 caracteres.

El conjunto de datos creado es **PTITPlates**, que fue etiquetado con la herramienta LabelMe, con imágenes de carreteras y zonas industriales con diferentes puntos de vista. El método propuesto logra alcanzar un **91.3%** de f1-score en el dataset PTITPlates y un **90.8** de f1-score en el dataset de Stanford Cars.

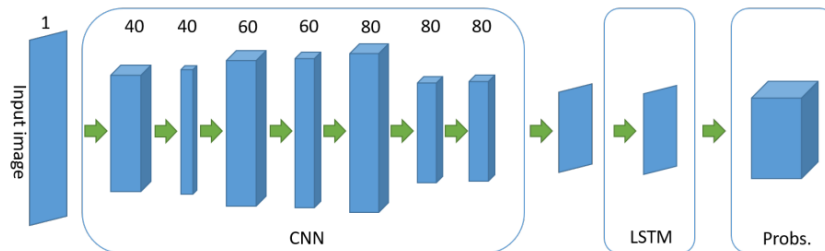


Figura 34: Arquitectura OCR
Fuente: <https://arxiv.org/pdf/2309.12972>

Los sistemas de identificación de matrículas requieren ser resilientes a cambios, debido al cambio constante y la cantidad constante de nuevas matrículas registradas al tráfico. Los métodos que usan transformadores exceden en este trabajo sin ningún problema, hasta recientemente, que su rendimiento no para de reducirse considerablemente. Florent Meyer et Al. [31] idean un transformador, **SaLT** que pueda identificar futuras matrículas con diferentes secuencias con el uso de varias modificaciones. El modelo base seleccionado es TrOCR²¹, el cual tiende a aprender la estructura exacta de las matrículas del conjunto de entrenamiento.

La primera propuesta es **cambiar el codificador del transformador por una FCN (Fully Convolutional Network)** para descartar la información posicional (intrínsecamente ligada al transformador); en vez de eso, esta FCN extrae las características de las imágenes sin este problema. Aquí surge otro problema: las redes convolucionales tienen problemas para capturar relaciones entre píxeles a largo plazo, especialmente al usar kernels de pequeño tamaño, como ResNet, así que **se reduce su uso hacia pequeños bloques que sólo se encarguen de retener la información y representación de un sólo caracter**, de modo que los rasgos de un caracter determinado no dependa de sus adyacentes.

La segunda modificación es en el decodificador, cuya parte no es eliminada (es necesaria para el reconocimiento de los caracteres), sino que se empieza por **eliminar los "token embeddings" del texto de la entrada y sustituirlos por una secuencia de codificaciones posicionales**. Lo siguiente es **eliminar estas últimas conexiones en la última capa**, de forma que no se dé el problema del desvanecimiento del gradiente y que además previene que justo antes de la capa de predicción se elimine la información de la posición. Gracias a eliminar la dependencia de la posición al finalizar, la última capa devuelve unas características sin posición.

Los conjuntos de datos propuestos son: un nuevo dataset sintético LPR-MNIST y el dataset LP. **LPR-MNIST** es una colección de 100,000 pares de imagen/texto concatenando 5 dígitos en blanco sobre fondo negro del dataset MNIST (cuyas imágenes fueron redimensionadas a 32 x 32 píxeles y seleccionadas aleatoriamente para cada dato del dataset, como se puede observar en la figura 35), siendo los números del dataset desde 00000 hasta 99999. El dataset **LP** contiene 797,469 y 97,725 para los conjuntos de entrenamiento y validación, respectivamente. Este nuevo transformador modificado logra alcanzar un **97.3%** de precisión en el dataset LP, contrastando de manera significativa con los 27.8% del transformador TrOCR.



Figura 35: Ejemplos del dataset LPR-MNIST

Fuente: <https://arxiv.org/pdf/2506.17051>

El coste computacional de las CNNs limitan su uso en sistemas de reconocimiento de

²¹El modelo es accesible por: https://huggingface.co/docs/transformers/model_doc/trocr

matrículas. El trabajo presentado por George B. Nardes et Al. [32], presenta una CNN de 8 bits cuantificados diseñada para el reconocimiento óptico de caracteres (OCR) de matrículas de Mercosur y Brasil. Los datasets empleados son SATTRA y UFPR-ALPR²², que juntos acaban siendo 82308/3500/13300 imágenes para cada conjunto, respectivamente, y una distribución de clases no balanceada (excepto en el conjunto de validación).

La arquitectura base seleccionada es LeNet-5, debido a su compactación y bajo número de parámetros, para poder terminar siendo un modelo muy eficiente y portátil cuya arquitectura se puede observar en la figura 36. Las modificaciones aplicadas fueron:

- La entrada fue modificada para aceptar **3 canales de 32 x 24 píxeles**.
- Las capas convolucionales se rediseñaron todas para que usen **kernels de 3 x 3**.
- Las 3 últimas capas totalmente conectadas se convierten en una única, reemplazando las dos primeras por una **capa convolucional y una capa de agrupación** (pooling). La última parte de la arquitectura termina siendo una cabeza FC de 128 x 35, siendo 35 el número de caracteres del conjunto de entrenamiento.

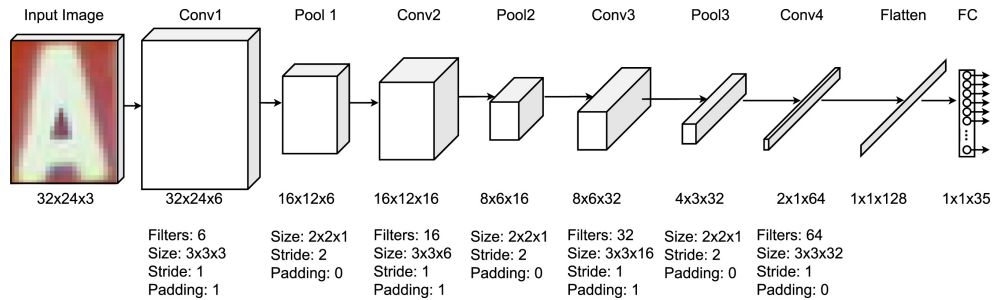


Figura 36: Arquitectura de LeNet-5 modificada

Fuente: <https://www.sciencedirect.com/science/article/pii/S016792602600009X>

Este modelo logra altos resultados de rendimiento en los datasets de UFPR, con una precisión de **90.22%** y una matriz de confusión de la figura 37, y sobre todo en el dataset SATTRA, con una precisión de **97.11%**. Comparando rendimiento, aunque se comparan varias GPUs, es notable la comparativa entre los **1.99ms** y **502 FPS** de la NVIDIA Titan XP con los **10.20ms** y **98 FPS** de la Tesla P100.

3 Métodos seleccionados

3.1 YOLOv26n

YOLO26 [33], publicado en Septiembre de 2025, es la versión más novedosa entre los modelos YOLO, con más mejoras en cuestión de eficiencia, precisión y despliegue en dispositivos de menor rendimiento; algo que se puede observar en la figura 38.

²²El dataset es accesible por: <https://github.com/raysonlaroca/ufpr-alpr-dataset>

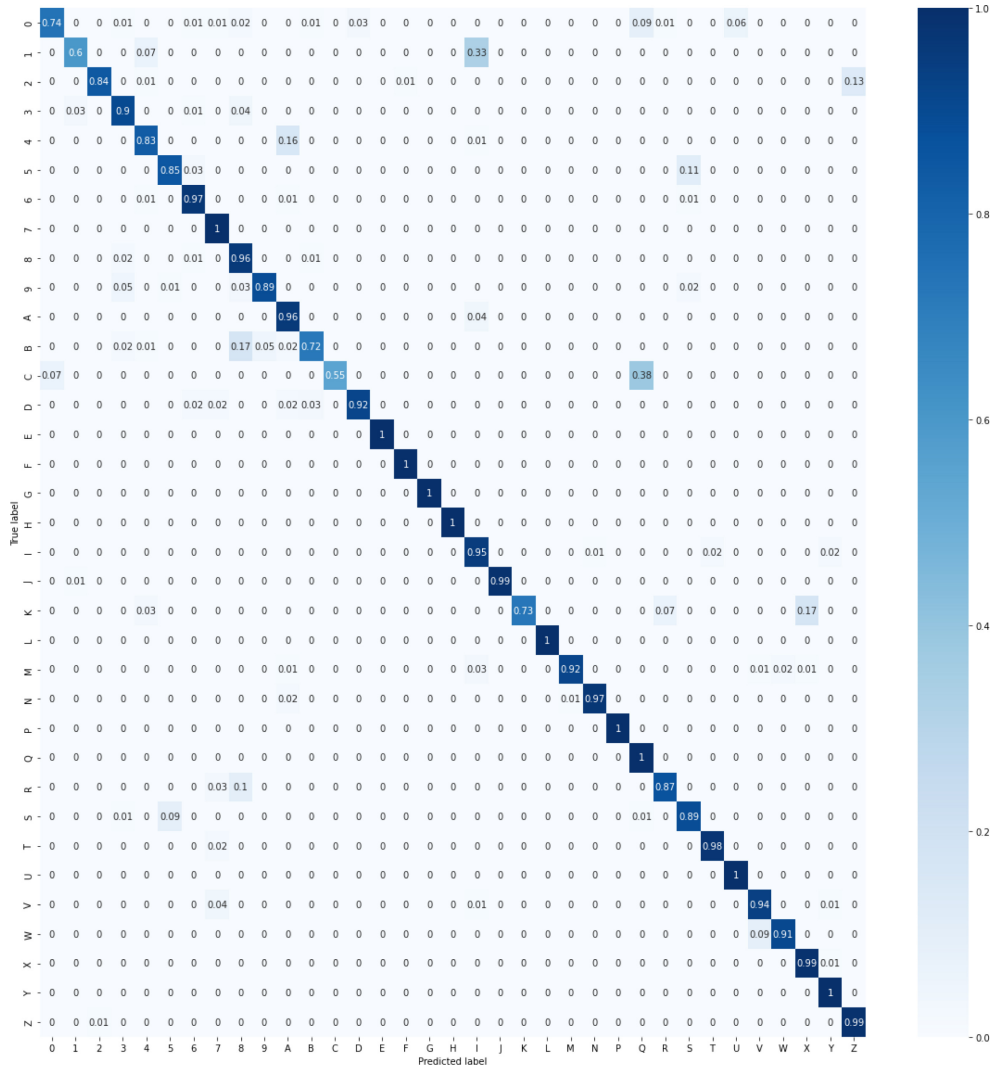


Figura 37: Arquitectura de LeNet-5 modificada

Fuente: <https://www.sciencedirect.com/science/article/pii/S016792602600009X>

3.1.1 Arquitectura de YOLO26

Esta nueva arquitectura, visible en la figura 39, elimina el *Distribution Focal Loss* (DFL)²³ e integrando [34]:

- **NMS-Free inference End to End:** NMS es una función que elimina *bounding boxes* superpuestas a otras, seleccionando la propuesta con mayor confianza. YOLO26 rediseña la predicción, haciendo que el modelo prediga **una sola bounding box por cada instancia** durante el entrenamiento. Eliminar esta parte **mejora en un 43% la latencia** en el entrenamiento con el uso de la CPU, en donde se hacía un cuello de botella debido a multitud de operaciones secuenciales.
- **ProgLoss:** Es una estrategia de pérdida de pesos dinámica. En otros detectores, tiene que haber un balance entre el arreglo por la función de pérdida de clasificación (L_{cls})

²³Función de pérdida que cuantifica el objetivo de la regresión en múltiples celdas o contenedores, permitiendo que sea más flexible la representación de las *bounding boxes*.

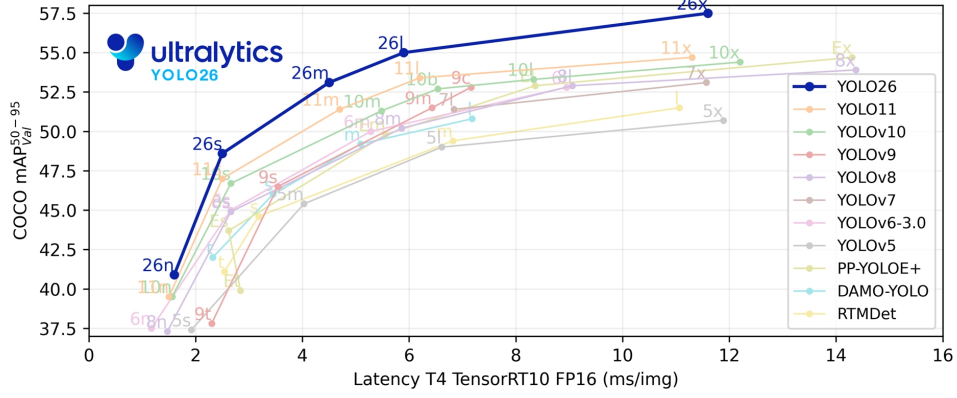


Figura 38: Compensación entre velocidad y precisión entre los distintos modelos YOLO

Fuente: <https://arxiv.org/pdf/2509.25164>

y el arreglo por la función de pérdida de regresión de la *bounding box* (L_{box}), lo que provoca que la red neuronal tenga que aprender a la vez a diferenciar entre características y a localizar con precisión sin el uso de anclajes.

Este nuevo sistema introduce un coeficiente λ_t en la ecuación:

$$L_{total}(t) = \lambda_t * L_{cls} + (1 - \lambda_t) * L_{box}$$

lo que resulta en que el aprendizaje prioriza las **características de alto nivel al principio** del entrenamiento y, **según progresa el aprendizaje**, este va priorizando **características geométricas de grano fino**, lo que se puede observar en la figura 40.

- **Small-Target-Aware Label Assignment:** STAL es una implementación que nace para solucionar el problema "desvanecimiento del objeto pequeño", donde la dependencia del umbral de IoU de estrategias típicas no funciona en objetos pequeños (ocupando menos de un 1% de la imagen) debido a la sensibilidad del IoU a pequeñas traslaciones espaciales. STAL, como se define en la siguiente ecuación:

$$\tau_{dynamic} = \tau_{base} * (1 - \alpha * \varepsilon^{-\frac{Area_{obj}}{Area_{img}}})$$

reemplaza el umbral fijo, que suele ser en torno a 0.5, por un umbral τ que se reduce según el tamaño del objeto y un parámetro α para la tasa de decaimiento en la que el umbral se reduce.

- **Optimizador MuSGD:** Debido a la eliminación del módulo DFL, es necesario introducir un entrenamiento más robusto que no permita que el descenso del gradiente colapse. Ultralytics introduce a YOLO26 el optimizador MuSGD, un optimizador híbrido que fusiona las propiedades del SGD estándar con el optimizador Muon. Este último realiza una ortogonalización matricial, actualizando la matriz de pesos para que sea ortogonal a su estado actual.

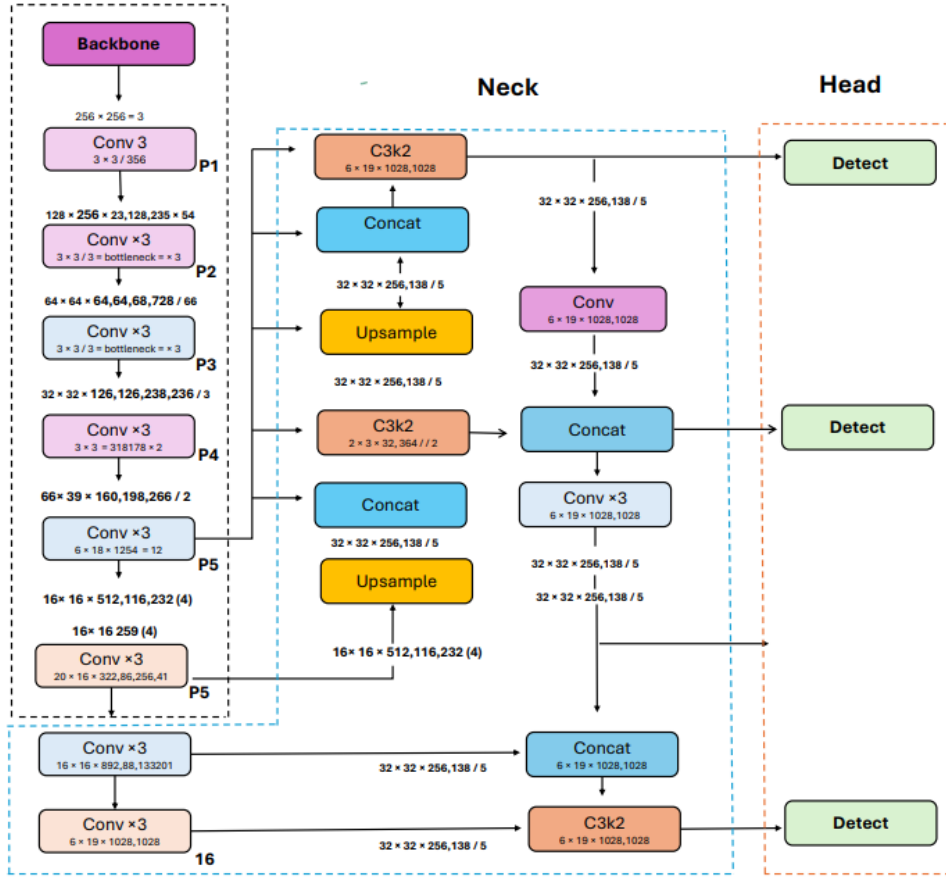


Figura 39: Arquitectura del modelo YOLO26 para detección y segmentación

Fuente: <https://arxiv.org/pdf/2509.25164>

Definimos el *buffer* del momento estándar v_t usado en SGD

$$v_{t+1} = \beta * v_t + g_t$$

donde g_t es el gradiente y β es el coeficiente del momento. MuSGD modifica la última actualización de los pesos usando la **ortogonalización de Newton-Schulz**²⁴, la cual "blanquea" la matriz de gradientes usando un proceso iterativo de refinamiento:

$$\theta_{t+1} = \theta_t - \eta * (\alpha * v_{t+1} + (1 - \alpha) * \text{NewtonSchulz}(g_t))$$

Este método **mitiga la varianza** de SGD mientras **elimina la inestabilidad de las actualizaciones ortogonales** de Muon: figura 41.

²⁴En lugar de expandir polinomios largos y repetir multiplicaciones sobre dimensiones extensas, se agrupa el proceso iterativo en una estructura unificada que evalúa la contribución real de cada potencia matricial. Al identificar y eliminar términos de escasa relevancia y al permitir que algunos coeficientes sean aprendidos por optimización, se consigue una combinación más parsimoniosa que converge de forma más estable y con menos operaciones pesadas [35].

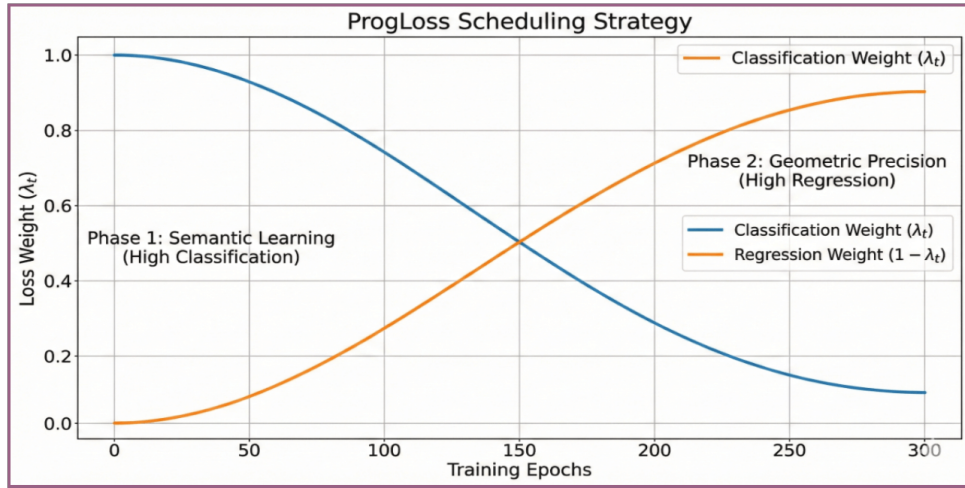


Figura 40: Estrategia de priorización del sistema Progloss
Fuente: <https://arxiv.org/html/2601.12882v1>

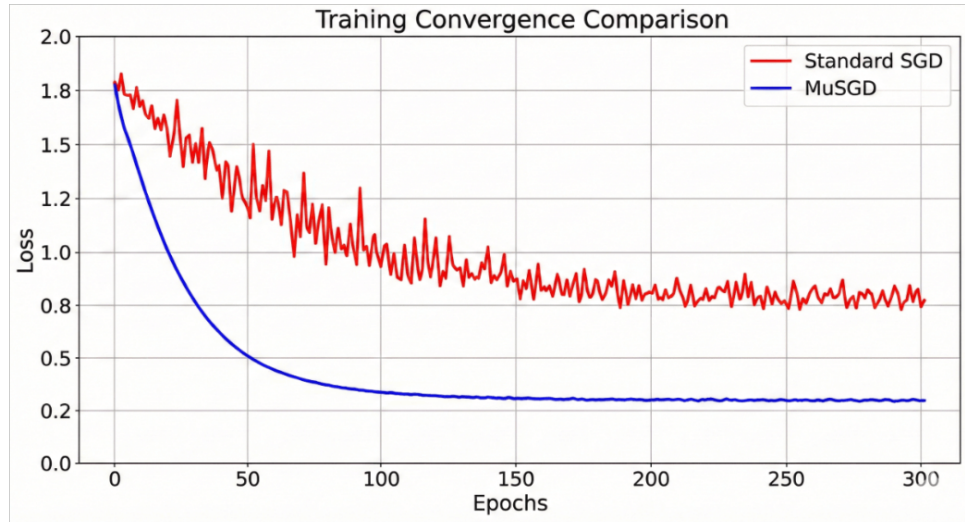


Figura 41: Concepto visual esperado de la optimización dinámica de MuSGD
Fuente: <https://arxiv.org/html/2601.12882v1>

3.1.2 Modelos

Aunque YOLO se creó con la principal finalidad de la tarea de detección, se han ido creando distintos modelos a lo largo de las versiones de YOLO con diferentes fines: clasificación, segmentación de instancias, detección de puntos claves/poses y detección de objetos orientados. Actualmente, Ultralytics construye 5 modelos distintos con la misma estructura, aunque con diferentes tamaños de capas. Para esta última versión de detección de YOLO, YOLO26, hay 5 modelos más o menos potentes (dependiendo del número de parámetros), cuyas especificaciones se pueden observar en la tabla 9 usando imágenes de 640 x 640 píxeles, redimensionadas del conjunto de datos COCO.

Modelo	mAP _{val50-95}	Velocidad T4 TRT10(ms)	Parámetros(M)	FLOPs(B)
YOLO26n	40.9%	1.7 ms	2.4 M	5.4 B
YOLO26s	48.6%	2.5 ms	9.5 M	20.7 B
YOLO26m	53.1%	4.7 ms	20.4 M	68.2 B
YOLO26l	55.0%	6.2 ms	24.8 M	86.4 B
YOLO26x	57.5%	11.8 ms	55.7 M	193.9 B

Tabla 9: Resultados de los distintos modelos YOLO26 de detección

3.2 RT-DETR

RT-DETR [36] es, como su propio nombre indica, el primer detector end-to-end en tiempo real: debido al impacto negativo del uso de la función NMS en los modelos YOLO (anteriores a YOLO26, el cual elimina esta función), estos modelos perdían precisión y velocidad. Estos detectores de objetos se caracterizan por un proceso optimizado cuya base de este algoritmo (DETR) [37] es un detector basado en un transformador que modela la detección de objetos como una tarea de predicción y asigna las etiquetas por emparejamiento de grafos bipartitos²⁵. A pesar de tener bastantes ventajas, DETR sufría de una convergencia en el entrenamiento muy lenta, un coste computacional alto y unas colas difíciles de optimizar; algo que los modelos DETR posteriores tratan de solucionar:

- **Acelerar la convergencia:** Deformable DETR [38] mejora la eficiencia del mecanismo de atención con características multiescala. DAB-DETR [39] y DN-DETR [40] introducen un esquema iterativo de refinamiento y un entrenamiento con supresión de ruido, mientras que Group-DETR [41] introduce una asignación *one-to-many* por grupos.
- **Reducción del coste computacional:** Efficient DETR [42] y Sparse DETR [43] reducen el número de capas del codificador-decodificador o el número de colas, mientras que Lite DETR [44] reduce la frecuencia de actualización de características de bajo nivel de forma intercalada.
- **Optimización de inicialización de las colas:** Conditional DETR [45] y Anchor DETR [46] reducen la dificultad computacional de las colas, Deformable DETR [38] propone un DETR de doble etapa con un selector de colas mixtas y DINO [47] aprovecha este mismo selector para inicializar de manera más eficaz las colas.

Este modelo, cuya arquitectura se puede observar en la figura 42, consiste en un extractor de características (backbone), un codificador híbrido eficiente y un decodificador de transformador con cabezas de predicción auxiliares:

- **Backbone:** Sirve como el extractor de características principal para transformar los píxeles de la imagen en mapas de características multiescala que el codificador puede refinar. Las arquitecturas que se han seleccionado para probarse junto al codificador-

²⁵Forma de emparejar vértices de dos conjuntos diferentes de forma que cada vértice esté conectado por un borde a 1 solo vértice del otro conjunto.

decodificador son ResNet50 y ResNet101 [48] como sustitutos de los modelos L y X de YOLO; para los modelos más ligeros, como los S y L de YOLO se modifican tanto el backbone (por ResNet34 y ResNet18) y además se modifican el número de capas en el codificador-decodificador.

- **Efficient Hybrid Encoder:** El mecanismo de atención de Deformable DETR reduce el coste computacional, pero incrementa la longitud de la secuencia de características multiescala lo que deriva en que el codificador se convierta en un cuello de botella, usando el 49% de los GFLOPs del modelo y contribuyendo solamente un 11% de AP. El nuevo codificador propuesto consiste en dos módulos:

- **Attention based Intra-scale Feature Interaction (AIFI):** AIFI reduce el coste computacional al ejecutar las interacciones intraescalares con un codificador de única escala sólo en la última etapa del **backbone**; en vez de aplicar este mecanismo de atención tanto en características de alto nivel (lo que captura la conexión entre entidades conceptuales y mejora el reconocimiento de objetos) como en las de bajo nivel, sólo se aplica en las de alto nivel, ya que es innecesario debido a la falta de entidades conceptuales y también para evitar la repetición de interacciones entre características con las de alto nivel.
- **CNN based Cross-scale Feature Fusion (CCFF):** CCFF se optimiza en base al módulo de fusión de escalas, que inserta varios bloques de fusión compuestos por capas convolucionales en la ruta de fusión. El bloque de fusión contiene dos convoluciones 1×1 , N **RepBlocks** [49] para fusionar características, y las salidas de las dos rutas se fusionan mediante suma elemento a elemento.

Teniendo en cuenta que las características de las 3 últimas etapas del **backbone** se conocen como S_3 , S_4 y S_5 y *Reshape* representa restaurar al mismo tamaño que S_5 , los cálculos del codificador se pueden resumir en: $Q = K = V = Flatten(S_5)F_5 = Reshape(AIFI(Q, K, V))O = CCFF(S_3, S_4, S_5)$

- **Decoder with Uncertainty-minimal Query Selection:** Para reducir la dificultad de optimizar las colas de objetos, anteriores trabajos proponían esquemas de selección de colas (proceso durante el cual se escogían las mejores características del codificador para inicializar las colas de objetos) que, al tratar de medir el rendimiento sobre una característica, incluía tanto la clasificación como la detección, lo que resultaba en inicializaciones subóptimas. En cambio, RT-DETR propone un esquema de selección de colas con mínima incertidumbre, que construye y optimiza la incertidumbre para modelar la unión latente de variables de las características del codificador (lo que proporciona al decodificador colas de alta calidad). Esta incertidumbre se construye a partir de la discrepancia entre las predicciones de la localización y las de clasificación.

Las comparaciones con modelos YOLO muestran resultados muy satisfactorios en una GPU T4 (ya que el propósito es usar un modelo ligero se compararán únicamente modelos

YOLO-S con el modelo RT-DETR que usa ResNet18) al entrenar RT-DETR en el dataset **Objects365** [50] y hacer *fine-tuning* en **COCO** con un entrenamiento de 72 iteraciones y alcanzando un **238/217 FPS** con los modelos con 2/3 capas del decodificador y 45.5%/46.5% de AP_{val} . En comparación, el modelo YOLOv8-S alcanza 136 FPS y 44.9% de AP_{val} ; esta leve mejora en la precisión aumenta considerablemente al usar los modelos RT-DETR más grandes, llegando a 52.2% de AP_{val} y 125 FPS con el modelo RT-DETR-R50 de 6 capas.

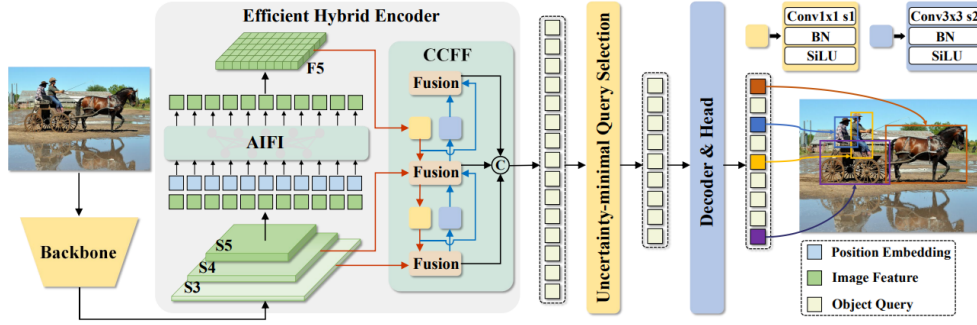


Figura 42: Vista general del modelo RT-DETR.

Fuente: <https://arxiv.org/html/2601.12882v1>

3.3 Tesseract OCR

Como intérprete y lector del texto de las matrículas, se ha seleccionado Tesseract OCR [51]. Este motor óptico de reconocimiento de caracteres de código abierto utiliza redes neuronales LSTM (*Long Short Term Memory*) para extraer el texto de las imágenes con un 99% de precisión en más de 100 idiomas.

4 Datasets

Se han investigado y recopilado varios conjuntos de datos desde varias fuentes, sobre todo desde Kaggle y Github. Los mejores datasets para el proyecto tendrían que tener tanto imágenes del mismo vehículo desde diferentes ángulos/cámaras como que la matrícula sea legible. Lamentablemente, el tipo de datasets que cumplan ambas funciones son bastante raros, aunque se han podido sacar 2 datasets que lo cumplen y otros dos que, aunque no hay posibilidad de variedad de cámaras hacia el mismo objetivo, aportan con gran cantidad de vehículos y sus matrículas.

4.1 Multi-perspective traffic video analysis

Este conjunto de datos [52] está comprendido por 3 escenarios distintos, las cuales a su vez se dividen en 3 perspectivas distintas (cámara del coche, dron y cámara de vigilancia), que a su vez cada una se divide en 2 secuencias de vídeo distintas grabadas a una resolución de **1920 x 1080 a 60 fotogramas por segundo**; el total de secuencias de vídeo alcanzando 18. Un ejemplo de fotograma del dataset se puede observar en la figura 43. Ya que este

conjunto de datos no lleva ninguna etiqueta asociada a alguno de los vídeos, se utilizará únicamente para el conjunto de pruebas.



Figura 43: Fotograma del escenario 1 con la perspectiva de una RSU
Fuente: <https://zenodo.org/records/18375218>

4.2 Bdd100k

Este conjunto de datos [53] es uno de los más grandes y diversos en cuestión de imágenes de vehículos con anotaciones precisas. Como sugiere el nombre, el dataset consta de 100,000 vídeos grabados desde una cámara acoplada en la parte delantera interior del vehículo; cada uno dura 40 segundos y están grabados a una resolución de **720 píxeles a 30 fps**. Para mantener un dataset diverso, Bdd100k cubre diferentes condiciones climáticas y periodos del día. Las anotaciones incluyen tanto las de detección como las de segmentación y cubren 10 categorías diferentes, como los autobuses y las señales, pudiéndose observar en la figura 44 el número de apariciones de cada una de ellas en el dataset. Un ejemplo del conjunto de datos se puede observar en la figura 45. Este conjunto de datos se utilizará inicialmente para entrenar y validar (para que el modelo se adapte bien a nuevos datos) los modelos.

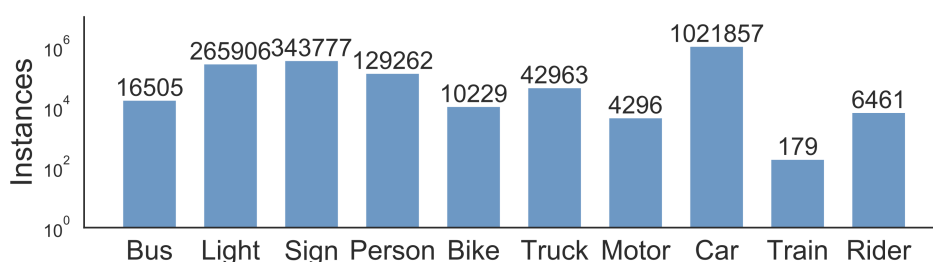


Figura 44: Estadísticas de los diferentes tipos de objetos en el dataset Bdd100k
Fuente: <https://bair.berkeley.edu/blog/2018/05/30/bdd/>



Figura 45: Ejemplo del dataset Bdd100k

Fuente: https://www.kaggle.com/datasets/solesensei/solesensei_bdd100k

4.3 UC3M-LP

Este conjunto de datos creado por investigadores de la universidad Carlos III de Madrid [54] es presentado para la tarea de **detección de matrículas y reconocimiento de caracteres** junto a otro dataset de gran tamaño para reidentificación de vehículos: UC3M-VRI. Se almacenan 1975 imágenes de 2547 vehículos distintos, lo que hacen 2547 matrículas, sumando un total de 12757 caracteres. Algunos ejemplos se pueden observar en la figura 46. La variedad de imágenes es amplia en cuestión de luz: 2185 imágenes de día y 362 de noche; la distancia de la cámara al vehículo es de entre 1 y 20 metros y la perspectiva cambia en cada imagen. El conjunto se ha probado en diferentes modelos de detección, incluyendo varias arquitecturas YOLO (varios modelos de los YOLOv3, YOLOv4 y YOLOv5), EfficientDet, ATSS y ASFF, siendo el resultado más notable el de **YOLO5s** con un tamaño de imagen de **1280 píxeles**, consiguiendo un **mAP@0.5 de 98.8%** y un **mAP@0.5-0.95 de 89.3%**. En el caso del reconocimiento de la matrícula, utilizando el mismo modelo con el tamaño de imagen de **640 píxeles** y el **optimizador AdamW** se consigue un **mAP@0.5 de 97.6%** y un **mAP@0.5-0.95 de 76.4%**.



Figura 46: Ejemplos del dataset UC3M-LP

Fuente: <https://github.com/ramajoballester/UC3M-LP>

4.4 UC3M-VRI

Este conjunto de datos es la otra mitad de la investigación del dataset anterior [54] y está orientado a reidentificación de vehículos. Se almacenan 1611 imágenes de 286 vehículos distintos todas sacadas de día y divididas en dos subconjuntos:

- **Conjunto de carretera:** 458 imágenes de 201 vehículos diferentes, sacadas a una distancia de entre 5 a 7 metros con una sola perspectiva.
- **Conjunto de intersección:** 1153 imágenes de 85 vehículos diferentes, sacadas a una distancia de entre 10 y 20 metros usando 2 cámaras para realizar diferentes planos sobre los vehículos que aparecen.

Algunos ejemplos se pueden observar en la figura 46. Los modelos seleccionados son EfficientNetB0 y FastReid, cada uno con otros 2 modelos que han sido entrenados en datasets diferentes: **Stanford-Cars** [55], **VeRi-776** y **VeRi-Wild**. El modelo con mayores resultados ha sido FastReid (entrenado en VeRi-776) con un 97.9% de accuracy con el test positivo-negativo (se selecciona una imagen (positiva) y se selecciona una segunda del resto de clases (negativa)) en el subdataset de carreteras y un 87.8% de accuracy en el subdataset de intersecciones.



Figura 47: Ejemplos del dataset UC3M-VRI
Fuente: <https://github.com/ramajoballester/UC3M-VRI>

5 Métricas

Para poder validar de forma equitativa los distintos métodos de detección, se tienen que seguir unas métricas a seguir para todos. Para empezar, se tiene que hablar del tiempo de procesamiento por imagen, ya que la velocidad de inferencia es prioritaria a la hora de detectar y reconocer claramente el vehículo y su matrícula.

- **mAP50:** Precisión media de las detecciones, calculada con un umbral de intersección (IoU) de 0,5.
- **mAP50-95:** Media entre todas las precisiones medias entre el umbral IoU 0,50 y 0,95, ambos umbrales incluidos.
- **Precisión:** En una clase, es el número de predicciones correctas para esta clase entre el número total de predicciones para esta misma clase.

- **Recall:** En una clase, es el número de predicciones correctas para esta clase entre el número total de instancias de esta misma clase.
- **F1-score:** Es una métrica especializada para cuando las clases están desbalanceadas, es decir, el número de instancias de cada clase no es el mismo para cada una. Se calcula primero para cada clase con la ecuación:

$$F1score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

En el caso de las métricas de Precision, Recall y F1-score, para hacer la media del dataset entero hay dos opciones: (1) **Macro:** El sumatorio del *f1-score* de cada clase entre el número de clases y (2) **Ponderada:** Considerar el número de instancias de cada clase, siendo las clases con mayor número de imágenes las que más aportan a la media.

Además

6 Metodología experimental

La metodología que se ha empleado es la habitual en el desarrollo de programas relacionados con *Machine Learning* y/o *Deep Learning*, similar al de la figura 48: (1) Recogida de datos, (2) Procesamiento de los datos, (3) Selección de los modelos, (4) Entrenamiento de los modelos, (5) Validación del modelo y resultados.

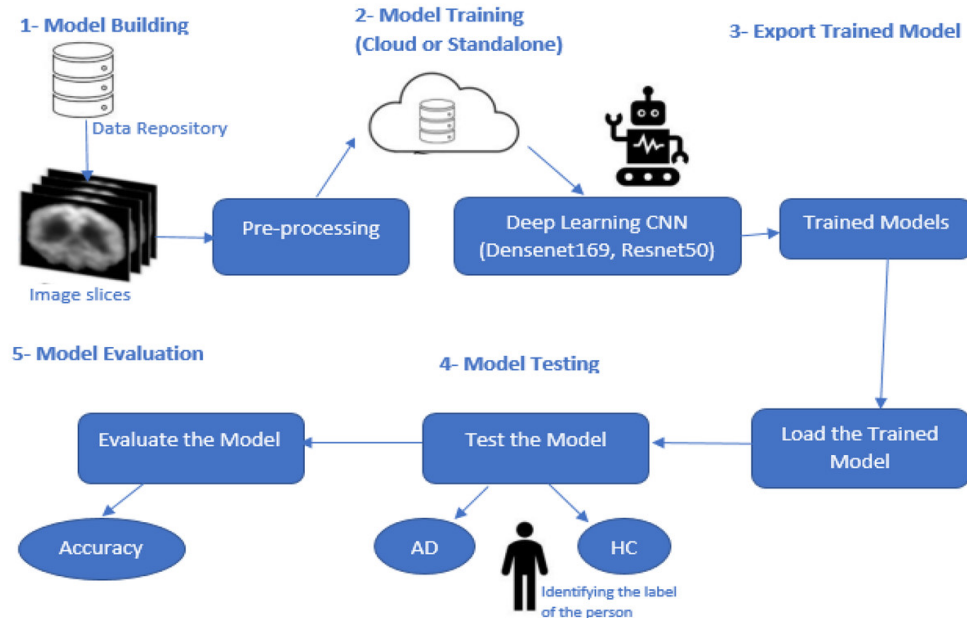


Figura 48: Metodología del diagnóstico y clasificación de Alzheimer con CNNs

Fuente: <https://peerj.com/articles/cs-1177/>

6.1 Preprocesado de datos

Ya que el modelo YOLO necesita un estructurado concreto, lo primero es construir programas y/o funciones personalizadas que modifiquen la estructura de los conjuntos de datos utilizados para entrenar y validar los modelos.

Los modelos YOLO requieren que un archivo con etiquetas para cada imagen con varias líneas (una por objeto) con el formato `¡class_id¡ ¡center_x¡ ¡center_y¡ ¡width¡ ¡height¡`, correspondiendo al número identificador de la clase (número entero comprendido entre 0 y el número de clases menos 1), las coordenadas x e y del centro de la *bounding box* y el ancho y altura de esta última. Estos cuatro últimos valores están normalizados al rango [0,1].

La estructura del conjunto de datos que tiene que seguir YOLO es de la siguiente forma:

```
├── st_yolo.py
├── yolo26n.py
├── models
├── datasets
│   └── UC3MLP
│       ├── train1.yaml
│       ├── test
│       ├── train
│       ├── val
│       │   ├── images
│       │   └── labels
```

6.2 Entrenamiento

En este apartado se mostrarán los diferentes parámetros y configuraciones aplicadas o que se han probado en los entrenamientos de los modelos YOLO y DINO-DETR.

6.2.1 YOLO

Gracias a Ultralytics²⁶, está a disposición de estudiantes e investigadores una librería entera con montones de recursos para poder entrenar, validar y probar los modelos YOLO. De entre todos los parámetros disponibles, para el entrenamiento de este modelo se han utilizado o experimentado con los siguientes:

- **Model:** Modelo que será entrenado.
- **Data:** Archivo de configuración del dataset con formato *.yaml*. En este archivo se fijan las rutas a las carpetas de entrenamiento, validación y pruebas, el número de clases y la asignación de IDs a cada clase. El archivo usado para entrenar el modelo YOLO se puede observar en la figura 49.
- **Epochs:** Cada época representa una pasada por el dataset entero.

²⁶Toda la información de la librería de Ultralytics es de código abierto y está disponible en <https://github.com/ultralytics/ultralytics>

- **Time:** Tiempo máximo del entrenamiento. Si este campo no está vacío, se usará en vez del parámetro de las iteraciones.
- **Patience:** Número de etapas que hay que esperar antes de parar el entrenamiento durante las cuales las métricas de validación no mejoran.
- **Batch:** El tamaño de lote significa el número de datos de entrenamiento procesadas por el modelo en una sola iteración. En vez de procesarse el dataset entero, este se divide en unos grupos de datos o *batches*.
- **Imgsz:** Tamaño de imagen usado en el entrenamiento. Si las imágenes del conjunto de datos no están a este tamaño se redimensionan antes de introducirlas en el modelo.
- **Device:** Especifica si se quiere usar la **CPU**, una única **GPU** o múltiples **GPUs** para maximizar la eficiencia del modelo y tardar mucho menos en este proceso.
- **Workers:** Número de hilos trabajadores para la carga de datos. Influye especialmente en la velocidad del procesamiento si se está usando la *GPU*.
- **Optimizer:** Escoger el optimizador para el entrenamiento entre **SGD**, **Adam**, **AdamW**, etc. Afecta a la estabilidad y a la velocidad de convergencia. Por defecto, se selecciona por sí solo en función del modelo.
- **Multiscale:** Permite redimensionar un paquete de imágenes (por ejemplo, un valor de 0.25 crearía imágenes de 0.75x a 1.25x).
- **Cos_lr:** Se utiliza un *scheduler* o programador de la tasa de aprendizaje coseno, ajustando la tasa de aprendizaje siguiendo una curva coseno a lo largo de los *epochs* para mejorar la convergencia.
- **Freeze:** Número de primeras capas del modelo a congelar para reducir el número de parámetros entrenables.
- **Lr0:** Tasa de aprendizaje inicial, influye en la rapidez con la que se actualizan los pesos del modelo.
- **Lrf:** Factor de la tasa de aprendizaje, que sirve junto a la tasa de aprendizaje inicial, para calcular la tasa de aprendizaje final de la forma: $lr_{final} = lr0 \times lrf$. Se usa en conjunto con el *scheduler* (como el coseno) para ajustar la tasa de aprendizaje a lo largo del entrenamiento.
- **Weight decay:** Valor de regularización L2, que penaliza los pesos grandes para evitar que el modelo se sobreajuste a los datos de entrenamiento (overfitting).
- **Dropout:** Tasa de pérdida de nodos del modelo para la regularización en tareas de clasificación, que evita el sobreajuste omitiendo aleatoriamente unidades durante el entrenamiento.
- **Cache:** Decide si se quiere guardar en memoria RAM o disco las imágenes del con-

junto de datos. Mejora la velocidad de entrenamiento a costa de un mayor uso de memoria.

- **Plots:** Dibujar gráficas de las métricas de entrenamiento y validación al acabar de entrenar el modelo. Además, se guardan varias imágenes con las predicciones hechas por el modelo YOLO para comprobar su rendimiento.
- **Exist_ok:** Decidir si se sobrescribe la carpeta donde se guardan los archivos de entrenamiento.
- **Save:** Decide si se guardan los pesos durante el entrenamiento para fijar puntos de control temporales. Esto permite resumir el entrenamiento a posteriori
- **Save_period:** Cada cuántas épocas se realiza el guardado de pesos.
- **Single_cls:** El modelo trata todas las clases del archivo de configuración como una sola. Es útil para centrar al modelo en solamente la tarea de detección.
- **Project:** Nombre del directorio donde se guardarán los resultados de entrenamiento.

```
train: '/home/ruben/clase/Practicas/datasets/bdd100k_yolo/train/images'
val: '/home/ruben/clase/Practicas/datasets/bdd100k_yolo/val/images'

nc: 6

names:
  0: car
  1: truck
  2: bus
  3: train
  4: motor
  5: bike
```

Figura 49: Ejemplo de archivo .yaml de configuración para entrenar un modelo YOLO

El entrenamiento de este modelo YOLO consta de:

```
results_val = model.train(data = "./datasets/vehicles/train1.yaml",
                           epochs = 50,
                           device = device,
                           workers = 4,
                           batch = 16,
                           patience = 25,
                           cos_lr = True,
                           optimizer = "auto",
                           pretrained = False,
                           weight_decay = 0.0001,
                           freeze = 0,
                           verbose = True,
                           plots = True,
```



```

        exist_ok = True,
        save_period = 25,
        name = "y26_1",
    )

```

6.3 DINO-DETR

6.4 Rastreo del Overfitting

Con el objetivo de evaluar el rendimiento del modelo y detectar posibles indicios de sobreajuste durante el entrenamiento, se ha realizado un seguimiento detallado de los valores de pérdida (`loss_rates`), los cuales se registraron durante el entrenamiento en cada iteración en los conjuntos de entrenamiento y validación. El análisis de estas gráficas permite observar el comportamiento del modelo y determinar si está memorizando los datos de entrenamiento en lugar de generalizar correctamente.

En los experimentos realizados se ha prestado especial atención a la diferencia entre las pérdidas tanto de entrenamiento como de validación ya que dependiendo del diferencial entre ellas podría llevar al sobreajuste. Estas pérdidas se extraen de un archivo csv creado automáticamente por el entrenamiento del modelo YOLO, que, a cada iteración extrae las siguientes métricas: **`epoch`, `time`, `train/box_loss`, `train/cls_loss`, `train/dfl_loss`, `metrics/precision(B)`, `metrics/recall(B)`, `metrics/mAP50(B)`, `metrics/mAP50-95(B)`, `val/box_loss`, `val/cls_loss` y `val/dfl_loss`**. Aunque es muy útil seguir la evolución de las métricas como `precision`, `recall` o `mAP50-95`, para rastrear signos de `overfitting` o `underfitting`. En la figuras 51 y 50 se pueden ver dos configuraciones diferentes de entrenamiento para la detección de vehículos: en la primera figura se utiliza el parámetro *multiscale*, que redimensiona las imágenes por cierto valor durante el entrenamiento, mientras que en la segunda no se utiliza este parámetro. De valores similares, ambas gráficas dotan de un problema: el `underfitting`; en ambas gráficas la pérdida de validación se mantiene inferior a la de entrenamiento, lo que significa que el modelo tenía demasiada penalización durante el entrenamiento.

7 Implementaciones en Python

Todo el desarrollo de las prácticas se subirá al siguiente repositorio de **GitHub**:

<https://github.com/RubenFdezGlez/MURIA-Practicas>.

Este repositorio incluirá, además del código necesario para la ejecución del sistema, un **README** detallado con información sobre la implementación y la guía de instalación de:

- Entrenamiento, validación y pruebas del modelo YOLO.
- Despliegue de la aplicación web para la detección de vehículos.

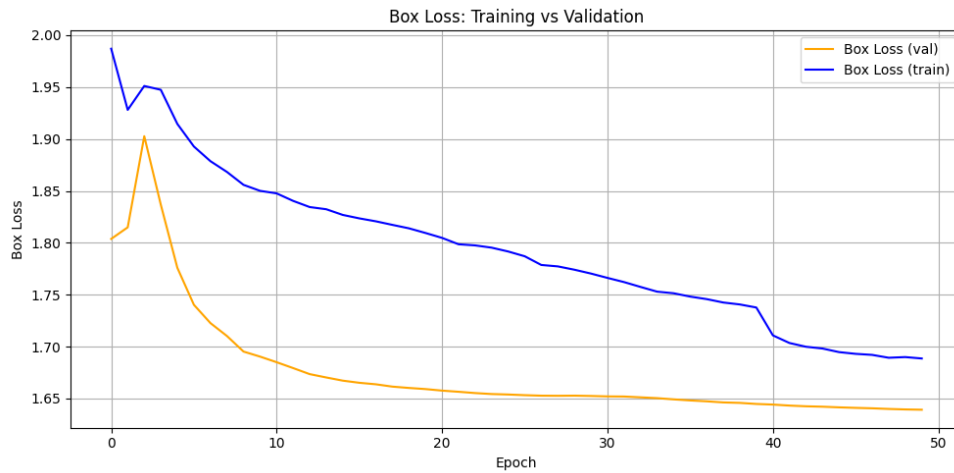


Figura 50: Gráfica de la evolución de las pérdida de detección de entrenamiento y validación sin multiscale

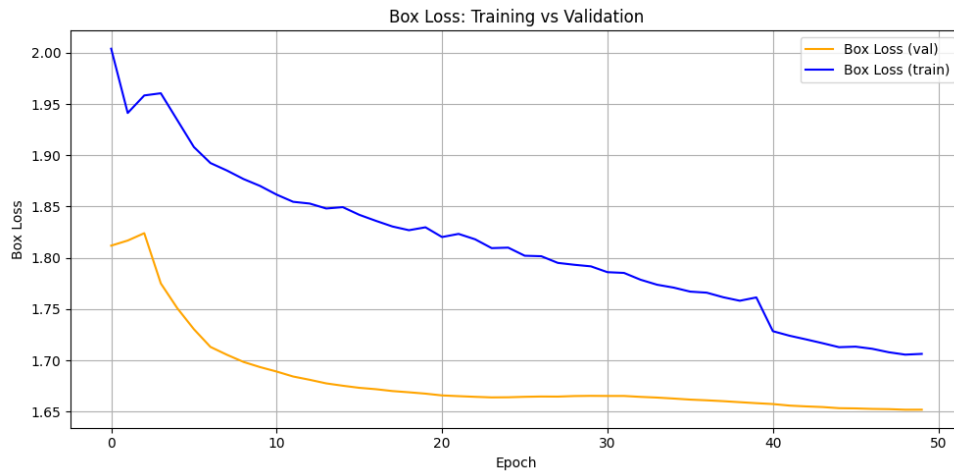


Figura 51: Gráfica de la evolución de las pérdida de detección de entrenamiento y validación con multiscale

8 CREDITS

References

- [1] Muhammad Umer Ramzan, Usman Ali, Syed Haider Abbas Naqvi, Zeeshan Aslam, Tehseen, Husnain Ali, and Muhammad Faheem. Enhancing vehicle entrance and parking management: Deep learning solutions for efficiency and security, 2023.
- [2] Anning Ji and Xintao Ma. Vehicle detection and classification for traffic management and autonomous systems using yolov10. *Alexandria Engineering Journal*, 127:804–816, 2025.
- [3] Leyre Encío, Daniel Fuertes, Carlos Del-Blanco, Iu Aguilar, Cristina Perez, Aleksandar Jevtić, Fernando Jaureguizar, and Narciso García. Enhanced nighttime vehicle

- detection for on-board processing. *IEEE Access*, PP:1–1, 01 2025.
- [4] K. Saiveena and P. Praveen. Edge intelligence-enabled vision transformer with yolov5 for accurate vehicle detection and image segmentation using adaptive multiscale deep learning mechanism. *Computers and Electrical Engineering*, 134:111086, 2026.
 - [5] Wenjie Zhu, Bushi Liu, Bolun Chen, Ling Wang, Xianchun Meng, Kailu Chang, and Jianyang Zhao. Dfem-net: A dynamic adaptive feature extraction network based deep learning model for pedestrian and vehicle detection. *Measurement*, 266:120336, 2026.
 - [6] Mohammed Alnusayri, Ghulam Mujtaba, Nouf Abdullah Almujaally, Shuo S. Aitarbi, Asaad Algarni, Ahmad Jalal, and Jeongmin Park. Traffic vision: Uav-based vehicle detection and traffic pattern analysis via deep learning classifier. *Computers, Materials and Continua*, 86(3), 2026.
 - [7] Seyed Hossein Hosseini DolatAbadi, Sayyed Mohammad Hossein Hashemi, Mohammad Hosseini, and Moein-Aldin AliHosseini. Revolutionizing traffic management with ai-powered machine vision: A step toward smart cities, 2025.
 - [8] Sara Beery, Guanhong Wu, Vivek Rathod, Ronny Votel, and Jonathan Huang. Context r-cnn: Long term temporal context for per-camera object detection, 2020.
 - [9] Sarvesh Shirulkar, Rishabhdev Makode, and Richa Khandelwal. Adaptive traffic signal management using real-time vehicle detection and tracking. In *2025 IEEE International Conference on Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI)*, volume 3, pages 1–6, 2025.
 - [10] B. Shyamala Gowri, Harini Harisitha S, Abishek Kanna M, and Amirthavarshini N S. You only look once version 8 (yolov8)-driven emergency vehicle detection and graph neural networks (gnns) based traffic signal prioritization. In *2025 5th International Conference on Trends in Material Science and Inventive Materials (ICTMIM)*, pages 1662–1667, 2025.
 - [11] Xiaohui Guan and Xinxin Sun. Vehicle identification and traffic statistics system in traffic intersection. In *2022 4th International Conference on Intelligent Control, Measurement and Signal Processing (ICMSP)*, pages 851–854, 2022.
 - [12] Chenyu Gu, Hong Du, Xiaozheng Zhang, Lidan Li, Zhonglin Yang, and Gaotian Liu. Yolov7 vehicle target detection algorithm based on attention mechanism. In *2024 IEEE 6th International Conference on Civil Aviation Safety and Information Technology (IC-CASIT)*, pages 349–352, 2024.
 - [13] Vaishnav V. Rathod, Rajesh Chouhan, Dipti P. Rana, Rupa G. Mehta, and Ashish Dhamaniya. Uav-based vehicle detection and classification under varying lighting conditions for indian traffic. In *2025 International Conference on Emerging Technology in Autonomous Aerial Vehicles (ETAAV)*, pages 1–5, 2025.
 - [14] Han Liu and Ronggui Ma. Full-view digital pavement reconstruction for practical

traffic scenarios through altitude-robust vehicle detection and traffic-adaptive optimization. *Automation in Construction*, 186:106893, 2026.

- [15] N.Jathin Reddy, L.R.P.L .Poojitha, M. Shashikanth, and Mohd Irfan. Yolov10-driven enhanced vehicle detection in low-light on-board environments. *International Journal of Scientific Research in Engineering and Management*, 10:1–9, 03 2026.
- [16] Liang Liang, Haihua Ma, Le Zhao, Xiaopeng Xie, Chengxin Hua, Miao Zhang, and Yonghui Zhang. Vehicle detection algorithms for autonomous driving: A review. *Sensors*, 24(10), 2024.
- [17] Hui Li, Peng Wang, Mingyu You, and Chunhua Shen. Reading car license plates using deep neural networks. *Image and Vision Computing*, 72:14–23, 2018.
- [18] Laixiang Xu, Zihan Shang, Xiangjun Chen, Tiwei Zeng, Wenhao Dai, and Junmin Zhao. Deep learning algorithms for license plate recognition: A review. *Neural Networks*, 200:108842, 2026.
- [19] Sai Sirisha Nadiminti, Pranav Kant Gaur, and Abhilash Bhardwaj. Exploration of an end-to-end automatic number-plate recognition neural network for indian datasets, 2022.
- [20] Natalya Kharina and Sergei Chernyadyev. Software for car license plates recognition with minimal computing resources. In *2022 24th International Conference on Digital Signal Processing and its Applications (DSPA)*, pages 1–4, 2022.
- [21] Hasan Aljzaere, Atul Chandra Nath, and Wolfram Hardt. German license plate recognition system using the yolov8 model and easyocr. In *2024 IEEE CPMT Symposium Japan (ICSJ)*, pages 176–179, 2024.
- [22] Emre Görgülü and Ahmet Remzi Özcan. Deep learning-based turkish license plate recognition system on low-power microcontroller systems. In *2024 8th International Artificial Intelligence and Data Processing Symposium (IDAP)*, pages 1–6, 2024.
- [23] Teófilo E. de Campos, Bodla Rakesh Babu, and Manik Varma. Character recognition in natural images. In *Proceedings of the Fourth International Conference on Computer Vision Theory and Applications - Volume 2: VISAPP, (VISIGRAPP 2009)*, pages 273–280. INSTICC, SciTePress, 2009.
- [24] Phayung Meesad and Wichan Thumthong. Advanced deep learning techniques for automated license plate recognition. *Scientific Reports*, 15(1):41194, Nov 2025.
- [25] Gulmini Pradhan, Manas Ranjan Prusty, Vipul Singh Negi, and Suchismita Chinara. Advanced iot-integrated parking systems with automated license plate recognition and payment management. *Scientific Reports*, 15(1):2388, Jan 2025.
- [26] Hanae Moussaoui, Nabil El Akkad, Mohamed Benslimane, Walid El-Shafai, Abdullah Baihan, Chaminda Hewage, and Rajkumar Singh Rathore. Enhancing automated vehicle identification by integrating yolo v8 and ocr techniques for high-precision li-

- cense plate detection and recognition. *Scientific Reports*, 14(1):14389, Jun 2024.
- [27] Dhiraj Neupane, Aashish Bhattarai, Sunil Aryal, Mohamed Reda Bouadjenek, Uk-Min Seok, and Jongwon Seok. Shine: Deep learning-based accessible parking management system, 2023.
 - [28] Nouar AlDahoul, Myles Joshua Toledo Tan, Raghava Reddy Tera, Hezerul Abdul Karim, Chee How Lim, Manish Kumar Mishra, and Yasir Zaki. Advancing vehicle plate recognition: Multitasking visual language models with vehiclepaligemma, 2024.
 - [29] Sergey Zherzdev and Alexey Gruzdev. Lprnet: License plate recognition via deep neural networks, 2018.
 - [30] Dat Tran-Anh, Khanh Linh Tran, and Hoai-Nam Vu. License plate recognition based on multi-angle view model, 2023.
 - [31] Florent Meyer, Laurent Guichard, Denis Coquenet, Guillaume Gravier, Yann Soullard, and Bertrand Couasnon. *Relaxed Syntax Modeling in Transformers for Future-Proof License Plate Recognition*, page 154–171. Springer Nature Switzerland, sept 2025.
 - [32] George B. Nardes, Thiago H. Rausch, Bruna H. Pereira, Douglas R. Melo, and Cesar A. Zeferino. A resource-constrained cnn accelerator for real-time license plate character recognition on fpga platforms. *Integration*, 108:102654, 2026.
 - [33] Ranjan Sapkota, Rahul Harsha Cheppally, Ajay Sharda, and Manoj Karkee. Yolo26: Key architectural enhancements and performance benchmarking for real-time object detection, 2026.
 - [34] Sudip Chakrabarty. Yolo26: An analysis of nms-free end to end framework for real-time object detection, 2026.
 - [35] UNSO: Ortogonalización Unificada de Newton Schulz — q2bstudio.com. <https://www.q2bstudio.com/nuestro-blog/606893/unificado-de-newton-schulz-metodo-de-ortogonalizacion-aplicado-en-calculos> [Accessed 09-04-2026].
 - [36] Yian Zhao, Wenyu Lv, Shangliang Xu, Jinman Wei, Guanzhong Wang, Qingqing Dang, Yi Liu, and Jie Chen. Detsr beat yolos on real-time object detection, 2024.
 - [37] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers, 2020.
 - [38] Xizhou Zhu, Weijie Su, Lewei Lu, Bin Li, Xiaogang Wang, and Jifeng Dai. Deformable detr: Deformable transformers for end-to-end object detection, 2021.
 - [39] Shilong Liu, Feng Li, Hao Zhang, Xiao Yang, Xianbiao Qi, Hang Su, Jun Zhu, and Lei Zhang. Dab-detr: Dynamic anchor boxes are better queries for detr, 2022.
 - [40] Feng Li, Hao Zhang, Shilong Liu, Jian Guo, Lionel M. Ni, and Lei Zhang. Dn-detr: Accelerate detr training by introducing query denoising, 2022.

- [41] Qiang Chen, Xiaokang Chen, Jian Wang, Shan Zhang, Kun Yao, Haocheng Feng, Junyu Han, Errui Ding, Gang Zeng, and Jingdong Wang. Group detr: Fast detr training with group-wise one-to-many assignment, 2023.
- [42] Zhuyu Yao, Jiangbo Ai, Boxun Li, and Chi Zhang. Efficient detr: Improving end-to-end object detector with dense prior, 2021.
- [43] Byungseok Roh, JaeWoong Shin, Wuhyun Shin, and Saehoon Kim. Sparse detr: Efficient end-to-end object detection with learnable sparsity, 2022.
- [44] Feng Li, Ailing Zeng, Shilong Liu, Hao Zhang, Hongyang Li, Lei Zhang, and Lionel M. Ni. Lite detr : An interleaved multi-scale encoder for efficient detr, 2023.
- [45] Depu Meng, Xiaokang Chen, Zejia Fan, Gang Zeng, Houqiang Li, Yuhui Yuan, Lei Sun, and Jingdong Wang. Conditional detr for fast training convergence, 2023.
- [46] Yingming Wang, Xiangyu Zhang, Tong Yang, and Jian Sun. Anchor detr: Query design for transformer-based object detection, 2022.
- [47] Hao Zhang, Feng Li, Shilong Liu, Lei Zhang, Hang Su, Jun Zhu, Lionel M. Ni, and Heung-Yeung Shum. Dino: Detr with improved denoising anchor boxes for end-to-end object detection, 2022.
- [48] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition, 2015.
- [49] Xiaohan Ding, Xiangyu Zhang, Ningning Ma, Jungong Han, Guiguang Ding, and Jian Sun. Repvgg: Making vgg-style convnets great again, 2021.
- [50] Shuai Shao, Zeming Li, Tianyuan Zhang, Chao Peng, Gang Yu, Jing Li, Xiangyu Zhang, and Jian Sun. Objects365: A large-scale, high-quality dataset for object detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 8425–8434, 2019.
- [51] Tesseract OCR Community. Tesseract OCR — The World’s Best Open Source OCR Engine — tesseractocr.org. <https://tesseractocr.org>. [Accessed 07-05-2026].
- [52] Ramon Sanchez-Iborra, Vasileios Kouvakis, Stylianos E. Trevlakis, Gonzalo Alarcon-Hellin, Theodoros Tsiftsis, Luis Bernal-Escobedo, Rodrigo Asensio-Garriga, Alexandros-Apostolos A. Boulogeorgos, and Antonio Skarmeta. Dataset for multi-perspective traffic video analysis. *Scientific Data*, Feb 2026.
- [53] Fisher Yu, Haofeng Chen, Xin Wang, Wenqi Xian, Yingying Chen, Fangchen Liu, Vashisht Madhavan, and Trevor Darrell. Bdd100k: A diverse driving dataset for heterogeneous multitask learning. In *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2020.
- [54] Álvaro Ramajo-Ballester, José María Armingol Moreno, and Arturo de la Escalera Hueso. Dual license plate recognition and visual features encoding for vehicle iden-

tification. *Robotics and Autonomous Systems*, 172:104608, 2024.

- [55] Jonathan Krause, Michael Stark, Jia Deng, and Li Fei-Fei. 3d object representations for fine-grained categorization. In *2013 IEEE International Conference on Computer Vision Workshops*, pages 554–561, Dec 2013.